baseline/model.py — 飞行精读关键段

Paper↔Code side-by-side

TAAC 2026 offline prep · 2026-05-04

> 配套文档：`docs/10-research/flight-reading-list-2026-05-06.md` · `docs/00-context/model-py-section-map.md`
> ★ 重点段落；★★ 核心必读；★★★ EXP-013 关键差异源头

目录

1. §1 ModelInput (数据结构) · L11-L19 · 对应 —
2. §2 RotaryEmbedding + rotate_half + apply_rope_to_tensor · L26-L92 · 对应 论文 2 RoFormer §3.4
3. §3 SwiGLU · L100-L114 · 对应 LLaMA/PaLM 系列惯例, baseline 默认激活
4. §4 RoPEMultiheadAttention ★ · L117-L243 · 对应 论文 1 Attention + 论文 2 RoFormer
5. §5 CrossAttention · L244-L314 · 对应 HyFormer Query Decoding
6. §6 RankMixerBlock ★ (Token Mixing 灵魂) · L315-L414 · 对应 论文 4 RankMixer

7. §7 MultiSeqQueryGenerator · L415-L557 · 对应 论文 3 HyFormer §3.4 Multi-Sequence
8. §8 SwiGLUEncoder (baseline 默认，3.5× 更快) · L558-L598 · 对应 —
9. §9 TransformerEncoder (对照) · L600-L671 · 对应 论文 1 Attention
10. §10 LongerEncoder · L672-L905 · 对应 论文 7 LONGER
11. §11 MultiSeqHyFormerBlock ★★ (HyFormer 核心 block) · L906-L1043 · 对应 论文 3 HyFormer §3.2-3.4
12. §12 OneTransMixedAttn ★★ (Mixed Parameterization) · L1044-L1217 · 对应 论文 5 OneTrans §Mixed Attn
13. §13 OneTransMixedFFN · L1218-L1280 · 对应 论文 5 OneTrans §Mixed FFN
14. §14 OneTransNSConditioner ★★★ (EXP-013 +0.010 AUC 源头) · L1281-L1349 · 对应 仓库自研扩展（有创新奖潜质）
15. §15 OneTransBlock · L1350-L1484 · 对应 论文 5 OneTrans (整体装配)
16. §16 GroupNSTokenizer (原 baseline 方式) · L1485-L1566 · 对应 —
17. §17 RankMixerNSTokenizer (当前默认) · L1567-L1688 · 对应 论文 4 RankMixer NS 部分
18. §18 PCVRHyFormer ★ (顶层装配) · L1689-L2396 · 对应 整体装配（所有论文的综合体现）

01. §1 ModelInput (数据结构)

行号范围：L11-L19（9 行） · **对应论文**：—

飞机上读的时候重点关注：

- 理解模型输入每个字段的 shape

```
11 | class ModelInput(NamedTuple):
12 |     user_int_feats: torch.Tensor
13 |     item_int_feats: torch.Tensor
14 |     user_dense_feats: torch.Tensor
15 |     item_dense_feats: torch.Tensor
16 |     seq_data: dict        # {domain: tensor [B, S, L]}
17 |     seq_lens: dict        # {domain: tensor [B]}
18 |     seq_time_buckets: dict  # {domain: tensor [B, L]}
19 |
```

02. §2 RotaryEmbedding + rotate_half + apply_rope_to_tensor

行号范围：L26-L92（67 行） · **对应论文**：论文 2 RoFormer §3.4

飞机上读的时候重点关注：

- inv_freq = 1/base^(2i/d) 即论文的 θ_i
- _build_cache 预计算 cos/sin
- rotate_half 是论文公式 (15) 的 pair-interleave 简化

```
26 | class RotaryEmbedding(nn.Module):
27 |     """Precomputes and caches RoPE cos/sin values.
28 |
29 |     Attributes:
30 |         dim: Rotary embedding dimension.
31 |         max_seq_len: Maximum sequence length for cache.
32 |         base: Base frequency for rotary encoding.
33 |     """
34 |
35 |     def __init__(self, dim: int, max_seq_len: int = 2048, base:
float = 10000.0) -> None:
36 |         super().__init__()
37 |         self.dim = dim
38 |         self.max_seq_len = max_seq_len
39 |         self.base = base
40 |
41 |         # Precompute inv_freq: (dim // 2,)
42 |         inv_freq = 1.0 / (base ** (torch.arange(0, dim,
2).float() / dim))
43 |         self.register_buffer('inv_freq', inv_freq,
persistent=False)
44 |
45 |         # Precompute cache
46 |         self._build_cache(max_seq_len)
47 |
48 |     def _build_cache(self, seq_len: int) -> None:
49 |         t = torch.arange(seq_len, dtype=self.inv_freq.dtype,
device=self.inv_freq.device)
50 |         freqs = torch.outer(t, self.inv_freq)  # (seq_len, dim //
2)
51 |         emb = torch.cat([freqs, freqs], dim=-1)  # (seq_len, dim)
52 |         self.register_buffer('cos_cached', emb.cos().unsqueeze(0),
persistent=False)  # (1, seq_len, dim)
53 |         self.register_buffer('sin_cached', emb.sin().unsqueeze(0),
persistent=False)  # (1, seq_len, dim)
54 |
55 |     def forward(self, seq_len: int, device: torch.device) ->
Tuple[torch.Tensor, torch.Tensor]:
56 |         """Computes cos/sin values for the given sequence length.
57 |
```

```
58 |         Returns pre-computed slices from the cache. The cache is
built once
59 |         in __init__ with max_seq_len; no runtime expansion is
performed so
60 |         that the forward pass remains compatible with
torch.compile().
61 |         """
62 |         cos = self.cos_cached[:, :seq_len, :].to(device)
63 |         sin = self.sin_cached[:, :seq_len, :].to(device)
64 |         return cos, sin
65 |
66 |
67 | def rotate_half(x: torch.Tensor) -> torch.Tensor:
68 |     """Swaps and negates the first and second halves of the last
dimension."""
69 |     x1 = x[..., : x.shape[-1] // 2]
70 |     x2 = x[..., x.shape[-1] // 2 :]
71 |     return torch.cat([-x2, x1], dim=-1)
72 |
73 |
74 | def apply_rope_to_tensor(
75 |     x: torch.Tensor,
76 |     cos: torch.Tensor,
77 |     sin: torch.Tensor,
78 | ) -> torch.Tensor:
79 |     """Applies Rotary Position Embedding to a single tensor.
80 |
81 |     Args:
82 |         x: (B, num_heads, L, head_dim)
83 |         cos: (1, L_max, head_dim) or (B, L, head_dim) for batch-
specific positions.
84 |         sin: Same shape as cos.
85 |
86 |     Returns:
87 |         Rotated tensor of shape (B, num_heads, L, head_dim).
88 |     """
89 |     L = x.shape[2]
90 |     cos_ = cos[:, :L, :].unsqueeze(1)  # (*, 1, L, head_dim)
91 |     sin_ = sin[:, :L, :].unsqueeze(1)
92 |     return x * cos_ + rotate_half(x) * sin_
```

03. §3 SwiGLU

行号范围：L100-L114（15 行）· **对应论文**：LLaMA/PaLM 系列惯例，baseline 默认激活

飞机上读的时候重点关注：

- 为什么 SiLU 比 ReLU/GELU 更稳定
- gate 后为什么再一次 Linear（fc_out）

```
100 | class SwiGLU(nn.Module):
101 |     """SwiGLU activation: x1 * SiLU(x2)."""
102 |
103 |     def __init__(self, d_model: int, hidden_mult: int = 4) ->
None:
104 |         super().__init__()
105 |         hidden_dim = d_model * hidden_mult
106 |         self.fc = nn.Linear(d_model, 2 * hidden_dim)
107 |         self.fc_out = nn.Linear(hidden_dim, d_model)
108 |
109 |     def forward(self, x: torch.Tensor) -> torch.Tensor:
110 |         x = self.fc(x)
111 |         x1, x2 = x.chunk(2, dim=-1)
112 |         x = x1 * F.silu(x2)
113 |         x = self.fc_out(x)
114 |         return x
```

04. §4 RoPEMultiheadAttention ★

行号范围：L117-L243（127 行）· **对应论文**：论文 1 Attention + 论文 2 RoFormer

飞机上读的时候重点关注：

- W_q/W_k/W_v/W_o 四投影 + W_g 是仓库扩展（gate）
- forward 里 reshape (B, L, D) → (B, h, L, d/h) 的维度变换
- RoPE 是在 Q/K 投影完成 reshape 后、dot-product 之前注入
- F.scaled_dot_product_attention 是 PyTorch 2.0+ 融合核

```
117 | class RoPEMultiheadAttention(nn.Module):
118 |     """Multi-head attention with Rotary Position Embedding
support.
119 |
120 |     Manually projects Q/K/V and reshapes for multi-head, then
injects RoPE
121 |     after projection and before dot-product. Uses
F.scaled_dot_product_attention
122 |     for efficient computation.
123 |     """
124 |
125 |     def __init__(
126 |         self,
127 |         d_model: int,
128 |         num_heads: int,
129 |         dropout: float = 0.0,
130 |         rope_on_q: bool = True,
131 |     ) -> None:
132 |         super().__init__()
133 |         self.d_model = d_model
134 |         self.num_heads = num_heads
135 |         self.head_dim = d_model // num_heads
```

```
136 |         self.rope_on_q = rope_on_q
137 |         self.dropout = dropout
138 |
139 |         assert d_model % num_heads == 0, "d_model must be
divisible by num_heads"
140 |
141 |         self.W_q = nn.Linear(d_model, d_model)
142 |         self.W_k = nn.Linear(d_model, d_model)
143 |         self.W_v = nn.Linear(d_model, d_model)
144 |         self.W_o = nn.Linear(d_model, d_model)
145 |         self.W_g = nn.Linear(d_model, d_model)
146 |
147 |         nn.init.zeros_(self.W_g.weight)
148 |         nn.init.constant_(self.W_g.bias, 1.0)
149 |
150 |     def forward(
151 |         self,
152 |         query: torch.Tensor,
153 |         key: torch.Tensor,
154 |         value: torch.Tensor,
155 |         key_padding_mask: Optional[torch.Tensor] = None,
156 |         attn_mask: Optional[torch.Tensor] = None,
157 |         rope_cos: Optional[torch.Tensor] = None,
158 |         rope_sin: Optional[torch.Tensor] = None,
159 |         q_rope_cos: Optional[torch.Tensor] = None,
160 |         q_rope_sin: Optional[torch.Tensor] = None,
161 |         need_weights: bool = False,
162 |     ) -> tuple:
163 |         """Computes multi-head attention with optional RoPE.
164 |
165 |         Args:
166 |             query: (B, Lq, D)
167 |             key: (B, Lk, D)
168 |             value: (B, Lk, D)
169 |             key_padding_mask: (B, Lk), True indicates padding
positions.
170 |             attn_mask: (Lq, Lk) or (B*num_heads, Lq, Lk),
additive mask.
171 |             rope_cos: (1, L, head_dim), RoPE for KV side (also
used for Q
172 |                 unless q_rope_* is provided).
173 |             rope_sin: Same shape as rope_cos.
174 |             q_rope_cos: (B, Lq, head_dim) or (1, Lq, head_dim),
Q-specific
175 |                 RoPE for cross-attention with gathered positions.
176 |             q_rope_sin: Same shape as q_rope_cos.
177 |             need_weights: Compatibility parameter, not used.
178 |
179 |         Returns:
180 |             Tuple of (output, None).
181 |         """
182 |         B, Lq, _ = query.shape
183 |         Lk = key.shape[1]
184 |
```

```
185 |         # 1. Linear projection
186 |         Q = self.W_q(query)  # (B, Lq, D)
187 |         K = self.W_k(key)    # (B, Lk, D)
188 |         V = self.W_v(value)  # (B, Lk, D)
189 |
190 |         # 2. Reshape to (B, num_heads, L, head_dim)
191 |         Q = Q.view(B, Lq, self.num_heads,
self.head_dim).transpose(1, 2)
192 |         K = K.view(B, Lk, self.num_heads,
self.head_dim).transpose(1, 2)
193 |         V = V.view(B, Lk, self.num_heads,
self.head_dim).transpose(1, 2)
194 |
195 |         # 3. Apply RoPE independently to Q and K
196 |         if rope_cos is not None and rope_sin is not None:
197 |             # K always uses rope_cos/rope_sin (KV-side positional
encoding)
198 |             K = apply_rope_to_tensor(K, rope_cos, rope_sin)
199 |
200 |             if self.rope_on_q:
201 |                 # Q side: prefer dedicated q_rope_cos/sin (top_k
positions in LongerEncoder cross-attn)
202 |                 q_cos = q_rope_cos if q_rope_cos is not None else
rope_cos
203 |                 q_sin = q_rope_sin if q_rope_sin is not None else
rope_sin
204 |                 Q = apply_rope_to_tensor(Q, q_cos, q_sin)
205 |
206 |         # 4. Convert key_padding_mask to SDPA format
207 |         sdpa_attn_mask = None
208 |         if key_padding_mask is not None:
209 |             # key_padding_mask: (B, Lk), True = padding
210 |             # SDPA expects (B, 1, 1, Lk) bool mask, True = attend
211 |             sdpa_attn_mask =
~key_padding_mask.unsqueeze(1).unsqueeze(2)  # (B, 1, 1, Lk)
212 |             sdpa_attn_mask = sdpa_attn_mask.expand(B,
self.num_heads, Lq, Lk)
213 |
214 |         if attn_mask is not None:
215 |             # attn_mask: additive float mask (Lq, Lk), -inf means
do not attend
216 |             # Convert to bool: positions that are not -inf are
True
217 |             bool_attn = (attn_mask == 0)  # (Lq, Lk)
218 |             bool_attn =
bool_attn.unsqueeze(0).unsqueeze(0).expand(B, self.num_heads, Lq, Lk)
219 |             if sdpa_attn_mask is not None:
220 |                 sdpa_attn_mask = sdpa_attn_mask & bool_attn
221 |             else:
222 |                 sdpa_attn_mask = bool_attn
223 |
224 |         # 5. Scaled Dot-Product Attention
225 |         dropout_p = self.dropout if self.training else 0.0
226 |         out = F.scaled_dot_product_attention(
```

```
            Q, K, V,
            attn_mask=sdpa_attn_mask,
            dropout_p=dropout_p,
        )  # (B, num_heads, Lq, head_dim)

        # Replace NaN from all-padding softmax with 0 (zero
vectors preserve original input via residual)
        out = torch.nan_to_num(out, nan=0.0)

        # 6. Reshape back and output projection
        out = out.transpose(1, 2).contiguous().view(B, Lq,
self.d_model)
        G = self.W_g(query)
        out = out * torch.sigmoid(G)
        out = self.W_o(out)

        return out, None


```

05. §5 CrossAttention

行号范围：L244-L314（71 行） · **对应论文**：HyFormer Query Decoding

飞机上读的时候重点关注：

- 和 self-attention 的唯一区别：Q 来源 vs K/V 来源不同
- HyFormer 中 Q = query tokens, K/V = sequence tokens

```
class CrossAttention(nn.Module):
    """Cross-attention module.

    Query comes from global tokens (Q tokens), Key/Value comes
from sequence
    tokens. Only applies RoPE to KV side (rope_on_q=False).
    """

    def __init__(
        self,
        d_model: int,
        num_heads: int,
        dropout: float = 0.0,
        ln_mode: str = 'pre'
    ) -> None:
        super().__init__()
        self.ln_mode = ln_mode

        self.attn = RoPEMultiheadAttention(
            d_model=d_model,
            num_heads=num_heads,
            dropout=dropout,
            rope_on_q=False,
```

```
        )

        if ln_mode in ['pre', 'post']:
            self.norm_q = nn.LayerNorm(d_model)
            self.norm_kv = nn.LayerNorm(d_model)

    def forward(
        self,
        query: torch.Tensor,
        key_value: torch.Tensor,
        key_padding_mask: Optional[torch.Tensor] = None,
        rope_cos: Optional[torch.Tensor] = None,
        rope_sin: Optional[torch.Tensor] = None,
    ) -> torch.Tensor:
        """Computes cross-attention between query tokens and
sequence tokens.

        Args:
            query: (B, Nq, D), query tokens.
            key_value: (B, L, D), sequence tokens.
            key_padding_mask: (B, L), True indicates padding
positions.
            rope_cos: (1, L, head_dim), KV-side RoPE cosine
values.
            rope_sin: (1, L, head_dim), KV-side RoPE sine values.

        Returns:
            Output tensor of shape (B, Nq, D).
        """
        residual = query

        if self.ln_mode == 'pre':
            query = self.norm_q(query)
            key_value = self.norm_kv(key_value)

        out, _ = self.attn(
            query=query,
            key=key_value,
            value=key_value,
            key_padding_mask=key_padding_mask,
            rope_cos=rope_cos,
            rope_sin=rope_sin,
        )

        out = residual + out

        if self.ln_mode == 'post':
            out = self.norm_q(out)

        return out

```

06. §6 RankMixerBlock ★ (Token Mixing 灵魂)

行号范围：L315-L414（100 行） · **对应论文**：论文 4 RankMixer

飞机上读的时候重点关注：

- token_mixing: (B,T,D)→(B,T,T,d_sub)→transpose→(B,T,D) 无参数
- ⚠️ 仓库用 shared FFN (fc1/fc2 各一份)，论文原版是 Per-token FFN
- 手画 T=4, d_sub=2 的 reshape 过程验证理解

```
315 | class RankMixerBlock(nn.Module):
316 |     """HyFormer Query Boosting block.
317 |
318 |     Performs three steps:
319 |     1. Token Mixing: Parameter-free tensor reshaping.
320 |     2. FFN: Two modes (see ``ffn_mode``).
321 |     3. Residual connection: Q_boost = Q + Q_e.
322 |
323 |     FFN modes (T19 / ADR-007):
324 |       - ``shared`` (default, back-compat): one ``nn.Linear`` per
layer, all
325 |         T tokens share the same FFN weights. Mirrors the legacy
behavior
326 |         of this block prior to T19.
327 |       - ``per_token``: each of the T tokens has its own
independent FFN
328 |         weight matrix. This matches the original RankMixer
paper's
329 |         "Per-token FFN" definition: allocating independent
parameters per
330 |         feature subspace to avoid dominant features flooding
long-tail
331 |         tokens. Parameters grow ~T× vs shared mode (T=15 @
d_model=64,
332 |         hidden_mult=4 → ~495k extra dense params per block).
333 |
334 |     Constraint: d_model must be divisible by n_total in 'full'
mode.
335 |     """
336 |
337 |     def __init__(
338 |         self,
339 |         d_model: int,
340 |         n_total: int,  # T = Nq + Nns
341 |         hidden_mult: int = 4,
342 |         dropout: float = 0.0,
343 |         mode: str = 'full',  # 'full' | 'ffn_only' | 'none'
344 |         ffn_mode: str = 'shared',  # 'shared' |
'per_token'  (T19 / ADR-007)
345 |     ) -> None:
346 |         super().__init__()
347 |         self.T = n_total
```

```
348 |         self.D = d_model
349 |         self.mode = mode
350 |         self.ffn_mode = ffn_mode
351 |         if ffn_mode not in ('shared', 'per_token'):
352 |             raise ValueError(
353 |                 f"ffn_mode must be 'shared' or 'per_token', got
{ffn_mode!r}"
354 |             )
355 |
356 |         if mode == 'none':
357 |             # Pure identity mapping, no submodules created
358 |             return
359 |
360 |         if mode == 'full':
361 |             if d_model % n_total != 0:
362 |                 raise ValueError(
363 |                     f"d_model={d_model} must be divisible by
T={n_total} for token mixing."
364 |                 )
365 |             self.d_sub = d_model // n_total
366 |
367 |         # FFN — used by both 'full' and 'ffn_only'.
368 |         self.norm = nn.LayerNorm(d_model)
369 |         self.dropout = nn.Dropout(dropout)
370 |         # Post-LN after residual to stabilize stacked block
outputs
371 |         self.post_norm = nn.LayerNorm(d_model)
372 |
373 |         hidden_dim = d_model * hidden_mult
374 |         self._hidden_dim = hidden_dim
375 |
376 |         if ffn_mode == 'shared':
377 |             # Legacy path: single Linear pair shared across all T
tokens.
378 |             self.fc1 = nn.Linear(d_model, hidden_dim)
379 |             self.fc2 = nn.Linear(hidden_dim, d_model)
380 |         else:
381 |             # Per-token FFN: independent weight matrices for
every token,
382 |             # implemented as grouped linear via torch.einsum so
it stays
383 |             # vectorized (no Python-level for-loop, torch.compile
friendly).
384 |             # Weight shapes:
385 |             #   fc1_weight: (T, D, hidden_dim)   fc1_bias: (T,
hidden_dim)
386 |             #   fc2_weight: (T, hidden_dim, D)   fc2_bias: (T, D)
387 |             # We do not use nn.ModuleList[Linear]*T because that
forces a
388 |             # Python for-loop on forward and defeats GPU
parallelism.
389 |             self.fc1_weight = nn.Parameter(torch.empty(n_total,
d_model, hidden_dim))
390 |             self.fc1_bias = nn.Parameter(torch.zeros(n_total,
```

```
hidden_dim))
391 |             self.fc2_weight = nn.Parameter(torch.empty(n_total,
hidden_dim, d_model))
392 |             self.fc2_bias = nn.Parameter(torch.zeros(n_total,
d_model))
393 |             # Mirror nn.Linear default initialization (Kaiming
uniform on
394 |             # weight, uniform bias in [-1/sqrt(fan_in), +1/
sqrt(fan_in)]).
395 |             # Doing it per-token keeps each token's FFN
statistically
396 |             # equivalent to a fresh nn.Linear at init.
397 |             for t in range(n_total):
398 |                 nn.init.kaiming_uniform_(self.fc1_weight[t],
a=math.sqrt(5))
399 |                 nn.init.kaiming_uniform_(self.fc2_weight[t],
a=math.sqrt(5))
400 |                 fan_in_1 = d_model
401 |                 fan_in_2 = hidden_dim
402 |                 bound_1 = 1.0 / math.sqrt(fan_in_1)
403 |                 bound_2 = 1.0 / math.sqrt(fan_in_2)
404 |                 nn.init.uniform_(self.fc1_bias[t], -bound_1,
bound_1)
405 |                 nn.init.uniform_(self.fc2_bias[t], -bound_2,
bound_2)
406 |
407 |     def token_mixing(self, Q: torch.Tensor) -> torch.Tensor:
408 |         """Performs parameter-free token mixing via reshape and
transpose.
409 |
410 |         Steps:
411 |         1. Splits channels into T subspaces: (B, T, D) -> (B, T,
T, d_sub).
412 |         2. Swaps token and subspace axes: (B, token, h, d_sub) ->
(B, h, token, d_sub).
413 |         3. Flattens back: (B, T, D).
414 |
```

07. §7 MultiSeqQueryGenerator

行号范围：L415-L557（143 行） · **对应论文**：论文 3 HyFormer §3.4 Multi-Sequence

飞机上读的时候重点关注：

- 每个 sequence 有 num_queries 个独立 FFN，生成 Q tokens
- GlobalInfo = Concat(NS 均池, Seq 均池)
- item_conditioned 分支是仓库 DIN-style 扩展（论文无）

```
415 |         Args:
416 |             Q: (B, T, D)
417 |
418 |         Returns:
419 |             Mixed tensor of shape (B, T, D).
420 |         """
421 |         B, T, D = Q.shape
422 |
423 |         # (B, T, D) -> (B, T, T, d_sub)
424 |         Q_split = Q.view(B, T, self.T, self.d_sub)
425 |
426 |         # (B, token, h, d_sub) -> (B, h, token, d_sub)
427 |         Q_rewired = Q_split.transpose(1, 2).contiguous()
428 |
429 |         # (B, T, T, d_sub) -> (B, T, D)
430 |         Q_hat = Q_rewired.view(B, T, D)
431 |         return Q_hat
432 |
433 |     def _apply_ffn(self, x: torch.Tensor) -> torch.Tensor:
434 |         """Apply FFN block. Dispatches to shared or per-token
path.
435 |
436 |         Args:
437 |             x: (B, T, D) normalized input.
438 |
439 |         Returns:
440 |             (B, T, D) FFN output (pre-residual).
441 |         """
442 |         if self.ffn_mode == 'shared':
443 |             h = self.fc1(x)
444 |             h = F.gelu(h)
445 |             h = self.dropout(h)
446 |             return self.fc2(h)
447 |         # per_token: each token indexes its own (D, hidden) /
(hidden, D) matrix.
448 |         # x: (B, T, D) · fc1_weight: (T, D, H) → (B, T, H)
449 |         # einsum does not allocate intermediate (B, T, D, H)
tensors.
450 |         h = torch.einsum('btd,tdh->bth', x, self.fc1_weight) +
self.fc1_bias
451 |         h = F.gelu(h)
452 |         h = self.dropout(h)
453 |         # (B, T, H) · fc2_weight: (T, H, D) → (B, T, D)
454 |         return torch.einsum('bth,thd->btd', h, self.fc2_weight) +
self.fc2_bias
455 |
456 |     def forward(self, Q: torch.Tensor) -> torch.Tensor:
457 |         """Applies query boosting: token mixing, FFN, and
residual connection.
458 |
459 |         Args:
460 |             Q: (B, T, D) where T = Nq + Nns.
461 |
462 |         Returns:
```

```
463 |             Boosted tensor of shape (B, T, D).
464 |         """
465 |         if self.mode == 'none':
466 |             return Q
467 |
468 |         # Token Mixing (parameter-free rewire) or identity
469 |         if self.mode == 'full':
470 |             Q_hat = self.token_mixing(Q)
471 |         else:  # 'ffn_only'
472 |             Q_hat = Q
473 |
474 |         # FFN block (shared or per-token)
475 |         x = self.norm(Q_hat)
476 |         Q_e = self._apply_ffn(x)
477 |
478 |         # Residual from original Q
479 |         Q_boost = Q + Q_e
480 |         Q_boost = self.post_norm(Q_boost)
481 |         return Q_boost
482 |
483 |
484 | class TargetItemSeqInjector(nn.Module):
485 |     """Inject target-item representation into each sequence token
(ADR-004).
486 |
487 |     Makes the per-sequence encoder target-aware by adding a per-
sequence
488 |     projection of the target item representation to every seq
token, before
489 |     the sequence encoder runs. This is orthogonal to the item-
conditioned
490 |     query branch in :class:`MultiSeqQueryGenerator` (which acts
on the query
491 |     side): injection here modifies the encoder's key/value input
so the
492 |     encoder itself can perform target-specific matching.
493 |
494 |     Mode ``'additive'`` (default):
495 |         target_repr = mean(item_ns_tokens)            # (B, D)
496 |         target_proj_i = LinearPerSeq_i(target_repr)   # (B, D)
497 |         seq_tokens_i' = seq_tokens_i + α_i *
target_proj_i.unsqueeze(1)
498 |
499 |     α_i is a learned per-sequence scalar initialised to
``alpha_init``
500 |     (default 0.0). With α=0 the injector is a functional no-op,
so training
501 |     starts equivalent to the baseline and the optimiser decides
the strength
502 |     of each domain's injection. This is the safest path to enable
target-
503 |     aware conditioning without breaking a well-trained baseline.
504 |
505 |     Padding tokens still receive the injection at the token
```

```
level, but the
506 |      downstream seq encoder masks them out via
``seq_padding_masks`` so they
507 |      contribute neither to pooling nor to attention output.
508 |
509 |      Mode ``'off'``: the module short-circuits (identity) and has
no params.
510 |      """
511 |
512 |      def __init__(
513 |          self,
514 |          d_model: int,
515 |          num_sequences: int,
516 |          mode: str = 'off',
517 |          alpha_init: float = 0.0,
518 |      ) -> None:
519 |          super().__init__()
520 |          self.d_model = d_model
521 |          self.num_sequences = num_sequences
522 |          self.mode = mode
523 |          if mode == 'off':
524 |              return
525 |          if mode != 'additive':
526 |              raise ValueError(
527 |                  f"Unsupported target_item_seq_injection mode:
{mode!r}. "
528 |                  f"Accepted: 'off', 'additive'.")
529 |          self.target_proj_per_seq = nn.ModuleList([
530 |              nn.Linear(d_model, d_model) for _ in
range(num_sequences)
531 |          ])
532 |          # Per-sequence learnable scalar; init=0 → starts as a no-
op so the
533 |          # optimiser decides how much each domain should attend to
target.
534 |          self.alpha = nn.Parameter(
535 |              torch.full((num_sequences,), float(alpha_init)))
536 |
537 |      def forward(
538 |          self,
539 |          seq_tokens_list: list,
540 |          item_repr: torch.Tensor,
541 |      ) -> list:
542 |          """Add α_i * W_i(item_repr) to each token of sequence i.
543 |
544 |          Args:
545 |              seq_tokens_list: List of (B, L_i, D) tensors, length
S.
546 |              item_repr: (B, D) target-item representation (mean of
item NS
547 |                  tokens, same source as ICQ).
548 |
549 |          Returns:
550 |              New list of (B, L_i, D) tensors with target injected.
```

```
Input list
551 |             is not mutated.
552 |         """
553 |         if self.mode == 'off':
554 |             return seq_tokens_list
555 |         out = []
556 |         for i in range(self.num_sequences):
557 |             # (B, D) → (B, 1, D), broadcast-add across sequence
length
```

08. §8 SwiGLUEncoder (baseline 默认，3.5× 更快)

行号范围：L558-L598（41 行） · **对应论文**：—

飞机上读的时候重点关注：

- 为什么它比 Transformer 快 3.5×（attention-free）
- Pre-LN + SwiGLU + 残差的极简结构

```
558 |             proj = self.target_proj_per_seq[i]
(item_repr).unsqueeze(1)
559 |             out.append(seq_tokens_list[i] + self.alpha[i] * proj)
560 |         return out
561 |
562 |
563 | class MultiSeqQueryGenerator(nn.Module):
564 |     """Multi-sequence query generation module.
565 |
566 |     Generates Q tokens independently for each sequence:
567 |     For each sequence i:
568 |         GlobalInfo_i = Concat(F1..FM, MeanPool(Seq_i))
569 |         Q_i = [FFN_{i,1}(GlobalInfo_i), ..., FFN_{i,N}
(GlobalInfo_i)]
570 |
571 |     When ``item_conditioned=True``, a per-sequence learned gate
blends the
572 |     standard global-info query with an item-aware branch:
573 |         item_repr   = mean(item_ns_tokens)        # (B, D)
574 |         gate_i      = σ(W_gate_i · [global_info_i, item_repr])  #
(B, D)
575 |         q_{i,j}     = (1 − gate_i) ⊙ FFN_{i,j}(global_info_i)
576 |                       +       gate_i  ⊙ FFN_item_{i,j}(item_repr)
577 |     This lets each domain independently learn how much the target
item
578 |     should steer its queries (DIN-style target-item awareness).
579 |     """
580 |
581 |     def __init__(
582 |         self,
583 |         d_model: int,
584 |         num_ns: int,
```

```
585 |         num_queries: int,
586 |         num_sequences: int,
587 |         hidden_mult: int = 4,
588 |         item_conditioned: bool = False,
589 |         num_item_ns: int = 0,
590 |     ) -> None:
591 |         super().__init__()
592 |         self.num_queries = num_queries
593 |         self.num_sequences = num_sequences
594 |         self.d_model = d_model
595 |         self.item_conditioned = item_conditioned
596 |         self.num_item_ns = num_item_ns
597 |
598 |         global_info_dim = (num_ns + 1) * d_model
```

09. §9 TransformerEncoder (对照)

行号范围：L600-L671（72 行） · **对应论文**：论文 1 Attention

飞机上读的时候重点关注：

- 标准 Pre-LN Transformer encoder layer
- 对比 SwiGLUEncoder 理解 attention 带来的代价

```
600 |         # LayerNorm on global_info to prevent gradient explosion
from large-dim concat
601 |         self.global_info_norm = nn.LayerNorm(global_info_dim)
602 |
603 |         # Each sequence has N independent FFNs
604 |         self.query_ffns_per_seq = nn.ModuleList([
605 |             nn.ModuleList([
606 |                 nn.Sequential(
607 |                     nn.Linear(global_info_dim, d_model *
hidden_mult),
608 |                     nn.SiLU(),
609 |                     nn.Linear(d_model * hidden_mult, d_model),
610 |                     nn.LayerNorm(d_model),
611 |                 )
612 |                 for _ in range(num_queries)
613 |             ])
614 |             for _ in range(num_sequences)
615 |         ])
616 |
617 |         if item_conditioned and num_item_ns > 0:
618 |             # Item-aware branch: per-sequence, per-query FFN
conditioned on item_repr
619 |             self.item_query_ffns_per_seq = nn.ModuleList([
620 |                 nn.ModuleList([
621 |                     nn.Sequential(
622 |                         nn.Linear(d_model, d_model *
hidden_mult),
```

```
623 |                             nn.SiLU(),
624 |                             nn.Linear(d_model * hidden_mult,
d_model),
625 |                             nn.LayerNorm(d_model),
626 |                         )
627 |                         for _ in range(num_queries)
628 |                     ])
629 |                     for _ in range(num_sequences)
630 |                 ])
631 |                 # Per-sequence gate: [global_info, item_repr] → D-dim
blend weight
632 |                 self.item_gates = nn.ModuleList([
633 |                     nn.Sequential(
634 |                         nn.Linear(global_info_dim + d_model,
d_model),
635 |                         nn.Sigmoid(),
636 |                     )
637 |                     for _ in range(num_sequences)
638 |                 ])
639 |
640 |     def forward(
641 |         self,
642 |         ns_tokens: torch.Tensor,
643 |         seq_tokens_list: list,
644 |         seq_padding_masks: list
645 |     ) -> list:
646 |         """Generates query tokens for each sequence.
647 |
648 |         Args:
649 |             ns_tokens: (B, M, D), shared NS tokens.
650 |             seq_tokens_list: List of (B, L_i, D) tensors, length
S.
651 |             seq_padding_masks: List of (B, L_i) masks, length S.
True
652 |                 indicates padding.
653 |
654 |         Returns:
655 |             List of (B, Nq, D) query token tensors, length S.
656 |         """
657 |         B = ns_tokens.shape[0]
658 |         ns_flat = ns_tokens.view(B, -1)  # (B, M*D)
659 |
660 |         # Extract item representation for item-conditioned
branch.
661 |         # item_ns tokens are the last num_item_ns tokens in
ns_tokens.
662 |         item_repr: Optional[torch.Tensor] = None
663 |         if self.item_conditioned and self.num_item_ns > 0:
664 |             item_repr = ns_tokens[:, -
self.num_item_ns:, :].mean(dim=1)  # (B, D)
665 |
666 |         q_tokens_list = []
667 |         for i in range(self.num_sequences):
668 |             # MeanPool(Seq_i)
```

```
669 |             valid_mask = ~seq_padding_masks[i]  # True = valid
670 |             valid_mask_expanded =
valid_mask.unsqueeze(-1).float()  # (B, L_i, 1)
671 |             seq_sum = (seq_tokens_list[i] *
valid_mask_expanded).sum(dim=1)  # (B, D)
```

10. §10 LongerEncoder

行号范围：L672-L905（234 行） · **对应论文**：论文 7 LONGER

飞机上读的时候重点关注：

- L > top_k: Cross-Attn，Q = 最新 top_k，K/V = 全部
- L ≤ top_k: Self-Attn
- 复杂度 $O(L^2) \rightarrow O(L_H \cdot L_S)$

```
672 |             seq_count =
valid_mask_expanded.sum(dim=1).clamp(min=1)  # (B, 1)
673 |             seq_pooled = seq_sum / seq_count  # (B, D)
674 |
675 |             # GlobalInfo_i = Concat(NS_flat, seq_pooled_i)
676 |             global_info = torch.cat([ns_flat, seq_pooled],
dim=-1)  # (B, (M+1)*D)
677 |             global_info = self.global_info_norm(global_info)
678 |
679 |             # Base queries from global info
680 |             base_queries = [ffn(global_info) for ffn in
self.query_ffns_per_seq[i]]
681 |
682 |             if item_repr is not None:
683 |                 # Item-aware branch: gate blends base query with
item-conditioned query
684 |                 gate = self.item_gates[i](
685 |                     torch.cat([global_info, item_repr], dim=-1))
# (B, D)
686 |                 item_queries = [
687 |                     ffn(item_repr) for ffn in
self.item_query_ffns_per_seq[i]]
688 |                 queries = [
689 |                     (1.0 - gate) * bq + gate * iq
690 |                     for bq, iq in zip(base_queries, item_queries)
691 |                 ]
692 |             else:
693 |                 queries = base_queries
694 |
695 |             q_tokens = torch.stack(queries, dim=1)  # (B, Nq, D)
696 |             q_tokens_list.append(q_tokens)
697 |
698 |         return q_tokens_list
699 |
700 |
```

```
# ============================================================================
# Sequence Encoders
# ============================================================================


class SwiGLUEncoder(nn.Module):
    """Efficient attention-free sequence encoder.

    Structure: x + Dropout(SwiGLU(LN(x))).
    """

    def __init__(
        self,
        d_model: int,
        hidden_mult: int = 4,
        dropout: float = 0.0
    ) -> None:
        super().__init__()
        self.norm = nn.LayerNorm(d_model)
        self.swiglu = SwiGLU(d_model, hidden_mult)
        self.dropout = nn.Dropout(dropout)

    def forward(
        self,
        x: torch.Tensor,
        key_padding_mask: Optional[torch.Tensor] = None,
        **kwargs
    ) -> torch.Tensor:
        """Applies the SwiGLU encoder with residual connection.

        Args:
            x: (B, L, D)
            key_padding_mask: (B, L), True indicates padding. Not
used by
                this encoder variant.
            **kwargs: Absorbs rope_cos/rope_sin and other unused
parameters.

        Returns:
            Tuple of (output tensor of shape (B, L, D),
key_padding_mask).
        """
        residual = x
        x = self.norm(x)
        x = self.swiglu(x)
        x = self.dropout(x)
        x = residual + x
        return x, key_padding_mask


class TransformerEncoder(nn.Module):
    """High-capacity sequence encoder with self-attention and
```

```
RoPE.
750 |
751 |     Structure: Standard Transformer Encoder Layer (Pre-LN).
752 |     """
753 |
754 |     def __init__(
755 |         self,
756 |         d_model: int,
757 |         num_heads: int,
758 |         hidden_mult: int = 4,
759 |         dropout: float = 0.0
760 |     ) -> None:
761 |         super().__init__()
762 |         self.norm1 = nn.LayerNorm(d_model)
763 |         self.norm2 = nn.LayerNorm(d_model)
764 |
765 |         self.self_attn = RoPEMultiheadAttention(
766 |             d_model=d_model,
767 |             num_heads=num_heads,
768 |             dropout=dropout,
769 |             rope_on_q=True,
770 |         )
771 |
772 |         hidden_dim = d_model * hidden_mult
773 |         self.ffn = nn.Sequential(
774 |             nn.Linear(d_model, hidden_dim),
775 |             nn.GELU(),
776 |             nn.Dropout(dropout),
777 |             nn.Linear(hidden_dim, d_model),
778 |             nn.Dropout(dropout)
779 |         )
780 |
781 |     def forward(
782 |         self,
783 |         x: torch.Tensor,
784 |         key_padding_mask: Optional[torch.Tensor] = None,
785 |         rope_cos: Optional[torch.Tensor] = None,
786 |         rope_sin: Optional[torch.Tensor] = None,
787 |     ) -> torch.Tensor:
788 |         """Applies one Transformer encoder layer.
789 |
790 |         Args:
791 |             x: (B, L, D)
792 |             key_padding_mask: (B, L), True indicates padding
positions.
793 |             rope_cos: (1, L, head_dim), RoPE cosine values.
794 |             rope_sin: (1, L, head_dim), RoPE sine values.
795 |
796 |         Returns:
797 |             Tuple of (output tensor of shape (B, L, D),
key_padding_mask).
798 |         """
799 |         # Self-Attention (Pre-LN) with RoPE
800 |         residual = x
```

```
801 |         x = self.norm1(x)
802 |         x, _ = self.self_attn(
803 |             query=x,
804 |             key=x,
805 |             value=x,
806 |             key_padding_mask=key_padding_mask,
807 |             rope_cos=rope_cos,
808 |             rope_sin=rope_sin,
809 |         )
810 |         x = residual + x
811 |
812 |         # FFN (Pre-LN)
813 |         residual = x
814 |         x = self.norm2(x)
815 |         x = self.ffn(x)
816 |         x = residual + x
817 |
818 |         return x, key_padding_mask
819 |
820 | class LongerEncoder(nn.Module):
821 |     """Top-K compressed sequence encoder.
822 |
823 |     Adapts behavior based on input length:
824 |     - L > top_k (first MultiSeqHyFormerBlock): Cross Attention.
825 |       Q = latest top_k tokens, K/V = all seq tokens -> output (B,
top_k, D).
826 |     - L <= top_k (subsequent MultiSeqHyFormerBlocks): Self
Attention.
827 |       Q = K = V = top_k tokens -> output (B, top_k, D).
828 |
829 |     Causal mask is only applied among top_k tokens (self-
attention layers);
830 |     the first cross-attention layer does not use a causal mask
since Q and K
831 |     have different lengths.
832 |
833 |     Returns (output, new_key_padding_mask) so downstream can
update the mask.
834 |     """
835 |
836 |     def __init__(
837 |         self,
838 |         d_model: int,
839 |         num_heads: int,
840 |         top_k: int = 50,
841 |         hidden_mult: int = 4,
842 |         dropout: float = 0.0,
843 |         causal: bool = False
844 |     ) -> None:
845 |         super().__init__()
846 |         self.top_k = top_k
847 |         self.causal = causal
848 |
849 |         # Pre-LN for attention
```

```
850 |         self.norm_q = nn.LayerNorm(d_model)
851 |         self.norm_kv = nn.LayerNorm(d_model)
852 |
853 |         # Shared RoPEMHA for both cross and self attention
854 |         self.attn = RoPEMultiheadAttention(
855 |             d_model=d_model,
856 |             num_heads=num_heads,
857 |             dropout=dropout,
858 |             rope_on_q=True,
859 |         )
860 |
861 |         # FFN (Pre-LN + residual)
862 |         self.ffn_norm = nn.LayerNorm(d_model)
863 |         hidden_dim = d_model * hidden_mult
864 |         self.ffn = nn.Sequential(
865 |             nn.Linear(d_model, hidden_dim),
866 |             nn.GELU(),
867 |             nn.Dropout(dropout),
868 |             nn.Linear(hidden_dim, d_model),
869 |             nn.Dropout(dropout)
870 |         )
871 |
872 |     def _gather_top_k(
873 |         self,
874 |         x: torch.Tensor,
875 |         key_padding_mask: torch.Tensor
876 |     ) -> Tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
877 |         """Selects the latest top_k valid tokens from each
sample.
878 |
879 |         Args:
880 |             x: (B, L, D)
881 |             key_padding_mask: (B, L), True indicates padding.
882 |
883 |         Returns:
884 |             top_k_tokens: (B, top_k, D)
885 |             new_padding_mask: (B, top_k), True indicates padding.
886 |             position_indices: (B, top_k), original position index
for each
887 |                 selected token, used for Q-side RoPE.
888 |         """
889 |         B, L, D = x.shape
890 |         device = x.device
891 |
892 |         # Valid lengths per sample
893 |         valid_len = (~key_padding_mask).sum(dim=1)  # (B,)
894 |
895 |         # Start position for each sample: max(valid_len - top_k,
0)
896 |         actual_k = torch.clamp(valid_len, max=self.top_k)  # (B,)
897 |         start_pos = valid_len - actual_k  # (B,)
898 |
899 |         # Build gather indices: (B, top_k)
900 |         offsets = torch.arange(self.top_k,
```

```
device=device).unsqueeze(0).expand(B, -1)  # (B, top_k)
901 |          indices = start_pos.unsqueeze(1) + offsets  # (B, top_k)
902 |
903 |          # For samples with valid_len < top_k, early indices may
exceed valid range;
904 |          # clamp to [0, L-1] and handle via mask below
905 |          indices = torch.clamp(indices, min=0, max=L - 1)
```

11. §11 MultiSeqHyFormerBlock ★★ (HyFormer 核心 block)

行号范围：L906-L1043（138 行） · **对应论文**：论文 3 HyFormer §3.2-3.4

飞机上读的时候重点关注：

- Step 1 Sequence Evolve: 每个序列独立 encoder
- Step 2 Query Decode: 每个 Q 集合 cross-attend 到对应序列
- Step 3 Query Boost: 所有 Q + NS 拼起来过 RankMixer
- n_total = num_queries * S + num_ns，决定 d_model 约束

```
 906 |
 907 |          # Gather: (B, top_k, D)
 908 |          indices_expanded = indices.unsqueeze(-1).expand(-1, -1,
D)  # (B, top_k, D)
 909 |          top_k_tokens = torch.gather(x, dim=1,
index=indices_expanded)
 910 |
 911 |          # New padding mask: first (top_k - actual_k) positions
are padding
 912 |          new_valid_len = actual_k  # (B,)
 913 |          pad_count = self.top_k - new_valid_len  # (B,)
 914 |          pos_indices = torch.arange(self.top_k,
device=device).unsqueeze(0)  # (1, top_k)
 915 |          new_padding_mask = pos_indices < pad_count.unsqueeze(1)
# (B, top_k)
 916 |
 917 |          # Zero out tokens at padding positions
 918 |          top_k_tokens = top_k_tokens *
(~new_padding_mask).unsqueeze(-1).float()
 919 |
 920 |          # position_indices for Q-side RoPE
 921 |          position_indices = indices  # (B, top_k)
 922 |
 923 |          return top_k_tokens, new_padding_mask, position_indices
 924 |
 925 |      def forward(
 926 |          self,
 927 |          x: torch.Tensor,
 928 |          key_padding_mask: Optional[torch.Tensor] = None,
```

```
 929 |         rope_cos: Optional[torch.Tensor] = None,
 930 |         rope_sin: Optional[torch.Tensor] = None,
 931 |     ) -> Tuple[torch.Tensor, torch.Tensor]:
 932 |         """Applies the LongerEncoder with adaptive cross/self
attention.
 933 |
 934 |         Args:
 935 |             x: (B, L, D), sequence tokens.
 936 |             key_padding_mask: (B, L), True indicates padding.
 937 |             rope_cos: (1, L, head_dim), RoPE cosine values
(length must cover
 938 |                 original sequence length L).
 939 |             rope_sin: (1, L, head_dim), RoPE sine values.
 940 |
 941 |         Returns:
 942 |             output: (B, top_k, D), compressed sequence.
 943 |             new_key_padding_mask: (B, top_k), updated padding
mask.
 944 |         """
 945 |         B, L, D = x.shape
 946 |
 947 |         if L > self.top_k:
 948 |             # === Cross Attention mode (first
MultiSeqHyFormerBlock) ===
 949 |             # 1. Extract latest top_k tokens as query
 950 |             q, new_mask, q_pos_indices = self._gather_top_k(x,
key_padding_mask)
 951 |
 952 |             # 2. Pre-LN
 953 |             q_normed = self.norm_q(q)
 954 |             kv_normed = self.norm_kv(x)
 955 |
 956 |             # 3. Build Q-side RoPE cos/sin by gathering from
global cos/sin at top_k positions
 957 |             q_rope_cos = None
 958 |             q_rope_sin = None
 959 |             if rope_cos is not None and rope_sin is not None:
 960 |                 # rope_cos: (1, L_max, head_dim), q_pos_indices:
(B, top_k)
 961 |                 head_dim = rope_cos.shape[2]
 962 |                 # Expand to batch dimension
 963 |                 cos_expanded = rope_cos.expand(B, -1, -1)  # (B,
L_max, head_dim)
 964 |                 sin_expanded = rope_sin.expand(B, -1, -1)
 965 |                 idx = q_pos_indices.unsqueeze(-1).expand(-1, -1,
head_dim)  # (B, top_k, head_dim)
 966 |                 q_rope_cos = torch.gather(cos_expanded, 1, idx)
# (B, top_k, head_dim)
 967 |                 q_rope_sin = torch.gather(sin_expanded, 1, idx)
 968 |
 969 |             # 4. Cross Attention (no causal mask since Q and K
have different lengths)
 970 |             attn_out, _ = self.attn(
 971 |                 query=q_normed,
```

```
                    key=kv_normed,
                    value=kv_normed,
                    key_padding_mask=key_padding_mask,  # Original
(B, L) mask
                    rope_cos=rope_cos,
                    rope_sin=rope_sin,
                    q_rope_cos=q_rope_cos,
                    q_rope_sin=q_rope_sin,
                )
                out = q + attn_out  # Residual based on q
            else:
                # === Self Attention mode (subsequent
MultiSeqHyFormerBlocks) ===
                new_mask = key_padding_mask

                # Pre-LN (Q and KV share norm_q)
                x_normed = self.norm_q(x)

                # Causal mask
                attn_mask = None
                if self.causal:
                    attn_mask =
nn.Transformer.generate_square_subsequent_mask(
                        L, device=x.device
                    )

                attn_out, _ = self.attn(
                    query=x_normed,
                    key=x_normed,
                    value=x_normed,
                    key_padding_mask=key_padding_mask,
                    attn_mask=attn_mask,
                    rope_cos=rope_cos,
                    rope_sin=rope_sin,
                )
                out = x + attn_out

            # FFN (Pre-LN + residual)
            residual = out
            out = self.ffn_norm(out)
            out = self.ffn(out)
            out = residual + out

            return out, new_mask


def create_sequence_encoder(
    encoder_type: str,
    d_model: int,
    num_heads: int = 4,
    hidden_mult: int = 4,
    dropout: float = 0.0,
    top_k: int = 50,
    causal: bool = False
```

```
1023 | ) -> nn.Module:
1024 |     """Creates a sequence encoder of the specified type.
1025 |
1026 |     Args:
1027 |         encoder_type: One of 'swiglu', 'transformer', or
'longer'.
1028 |         d_model: Model dimension.
1029 |         num_heads: Number of attention heads (used by
transformer/longer).
1030 |         hidden_mult: FFN expansion multiplier.
1031 |         dropout: Dropout rate.
1032 |         top_k: Compression length for LongerEncoder (only used
by longer).
1033 |         causal: Whether to use causal mask in LongerEncoder
(only used by
1034 |             longer).
1035 |
1036 |     Returns:
1037 |         A sequence encoder module.
1038 |     """
1039 |     if encoder_type == 'swiglu':
1040 |         return SwiGLUEncoder(d_model, hidden_mult, dropout)
1041 |     elif encoder_type == 'transformer':
1042 |         return TransformerEncoder(d_model, num_heads,
hidden_mult, dropout)
1043 |     elif encoder_type == 'longer':
```

12. §12 OneTransMixedAttn ★★ (Mixed Parameterization)

行号范围：L1044-L1217（174 行） · **对应论文**：论文 5 OneTrans §Mixed Attn

飞机上读的时候重点关注：

- W_Q_s/W_K_s/W_V_s/W_O_s: S-tokens 共享 $4\times d^2$ 参数
- W_Q_ns[j]/…/W_O_ns[j]: 每个 NS token 独立 $4\times d^2\times M$ 参数
- ⚠️ 仓库 S-S 用 BIDIRECTIONAL 不用 causal（多 domain 按字母序拼，无时序）
- _build_causal_mixed_mask: 手画 L=4, M=3 的 7x7 mask 矩阵

```
1044 |         return LongerEncoder(d_model, num_heads, top_k,
hidden_mult, dropout, causal)
1045 |     else:
1046 |         raise ValueError(f"Unknown encoder type:
{encoder_type}")
1047 |
1048 |
1049 | #
═══════════════════════════════════════════════════════════════
1050 | # HyFormer Blocks
```

```
1051 | #
     ================================================================================
1052 |
1053 |
1054 | class MultiSeqHyFormerBlock(nn.Module):
1055 |     """Multi-sequence HyFormer block.
1056 |
1057 |     Each of the S sequences independently performs Sequence
Evolution and
1058 |     Query Decoding, then all Q tokens and shared NS tokens are
merged for
1059 |     joint Query Boosting.
1060 |     """
1061 |
1062 |     def __init__(
1063 |         self,
1064 |         d_model: int,
1065 |         num_heads: int,
1066 |         num_queries: int,
1067 |         num_ns: int,
1068 |         num_sequences: int,
1069 |         seq_encoder_type: str = 'swiglu',
1070 |         hidden_mult: int = 4,
1071 |         dropout: float = 0.0,
1072 |         top_k: int = 50,
1073 |         causal: bool = False,
1074 |         rank_mixer_mode: str = 'full',
1075 |         rank_mixer_ffn_mode: str = 'shared',
1076 |     ) -> None:
1077 |         super().__init__()
1078 |         self.num_sequences = num_sequences
1079 |         self.num_queries = num_queries
1080 |         self.num_ns = num_ns
1081 |
1082 |         # Independent sequence encoder per sequence
1083 |         self.seq_encoders = nn.ModuleList([
1084 |             create_sequence_encoder(
1085 |                 encoder_type=seq_encoder_type,
1086 |                 d_model=d_model,
1087 |                 num_heads=num_heads,
1088 |                 hidden_mult=hidden_mult,
1089 |                 dropout=dropout,
1090 |                 top_k=top_k,
1091 |                 causal=causal
1092 |             )
1093 |             for _ in range(num_sequences)
1094 |         ])
1095 |
1096 |         # Independent cross-attention per sequence
1097 |         self.cross_attns = nn.ModuleList([
1098 |             CrossAttention(
1099 |                 d_model=d_model,
1100 |                 num_heads=num_heads,
1101 |                 dropout=dropout,
```

```
1102 |                 ln_mode='pre'
1103 |             )
1104 |             for _ in range(num_sequences)
1105 |         ])
1106 |
1107 |         # RankMixer: input token count = Nq * S + Nns
1108 |         n_total = num_queries * num_sequences + num_ns
1109 |         self.mixer = RankMixerBlock(
1110 |             d_model=d_model,
1111 |             n_total=n_total,
1112 |             hidden_mult=hidden_mult,
1113 |             dropout=dropout,
1114 |             mode=rank_mixer_mode,
1115 |             ffn_mode=rank_mixer_ffn_mode,
1116 |         )
1117 |
1118 |     def forward(
1119 |         self,
1120 |         q_tokens_list: list,
1121 |         ns_tokens: torch.Tensor,
1122 |         seq_tokens_list: list,
1123 |         seq_padding_masks: list,
1124 |         rope_cos_list: Optional[List[torch.Tensor]] = None,
1125 |         rope_sin_list: Optional[List[torch.Tensor]] = None,
1126 |     ) -> Tuple[list, torch.Tensor, list, list]:
1127 |         """Processes one multi-sequence HyFormer block step.
1128 |
1129 |         Args:
1130 |             q_tokens_list: List of (B, Nq, D) tensors, length S.
1131 |             ns_tokens: (B, Nns, D)
1132 |             seq_tokens_list: List of (B, L_i, D) tensors, length
S.
1133 |             seq_padding_masks: List of (B, L_i) masks, length S.
1134 |             rope_cos_list: List of (1, L_i, head_dim) tensors,
length S.
1135 |             rope_sin_list: List of (1, L_i, head_dim) tensors,
length S.
1136 |
1137 |         Returns:
1138 |             A tuple (next_q_list, next_ns, next_seq_list,
next_masks), where
1139 |             next_q_list is a list of (B, Nq, D) updated query
tensors,
1140 |             next_ns is (B, Nns, D) updated non-sequence tokens,
1141 |             next_seq_list is a list of (B, L_i', D) encoded
sequence tensors,
1142 |             and next_masks is a list of (B, L_i') updated
padding masks.
1143 |         """
1144 |         S = self.num_sequences
1145 |         Nq = self.num_queries
1146 |
1147 |         # 1. Independent Sequence Evolution per sequence
1148 |         next_seqs = []
```

```
1149 |         next_masks = []
1150 |         for i in range(S):
1151 |             rc = rope_cos_list[i] if rope_cos_list is not None
else None
1152 |             rs = rope_sin_list[i] if rope_sin_list is not None
else None
1153 |             result = self.seq_encoders[i](
1154 |                 seq_tokens_list[i], seq_padding_masks[i],
1155 |                 rope_cos=rc, rope_sin=rs,
1156 |             )
1157 |             next_seq_i, mask_i = result
1158 |             next_seqs.append(next_seq_i)
1159 |             next_masks.append(mask_i)
1160 |
1161 |         # 2. Independent Query Decoding per sequence
1162 |         decoded_qs = []
1163 |         for i in range(S):
1164 |             rc = rope_cos_list[i] if rope_cos_list is not None
else None
1165 |             rs = rope_sin_list[i] if rope_sin_list is not None
else None
1166 |             decoded_q_i = self.cross_attns[i](
1167 |                 q_tokens_list[i], next_seqs[i], next_masks[i],
1168 |                 rope_cos=rc, rope_sin=rs,
1169 |             )
1170 |             decoded_qs.append(decoded_q_i)
1171 |
1172 |         # 3. Token Fusion: concatenate all decoded_q + ns_tokens
1173 |         combined = torch.cat(decoded_qs + [ns_tokens], dim=1)  #
(B, Nq*S + Nns, D)
1174 |
1175 |         # 4. Query Boosting
1176 |         boosted = self.mixer(combined)  # (B, Nq*S + Nns, D)
1177 |
1178 |         # 5. Split back into per-sequence Q and NS
1179 |         next_q_list = []
1180 |         offset = 0
1181 |         for i in range(S):
1182 |             next_q_list.append(boosted[:, offset:offset +
Nq, :])
1183 |             offset += Nq
1184 |         next_ns = boosted[:, offset:, :]
1185 |
1186 |         return next_q_list, next_ns, next_seqs, next_masks
1187 |
1188 |
1189 | #
================================================================
1190 | # OneTrans: Mixed-Parameterization Unified Attention Block
1191 | #
================================================================
1192 |
1193 |
1194 | class OneTransMixedAttn(nn.Module):
```

```
1195 |     """Mixed-Parameterization Multi-Head Attention for OneTrans.
1196 |
1197 |     Sequence layout: [S_token_0 ... S_token_{L-1} |
NS_token_0 ... NS_token_{M-1}]
1198 |
1199 |     Attention mask (True = can attend):
1200 |     - S_i  → S_0..S_i (causal), S cannot see NS
1201 |     - NS_j → all S (full), NS_0..NS_j (causal within NS)
1202 |
1203 |     Mixed parameterization:
1204 |     - S positions: shared W_Q^s, W_K^s, W_V^s, W_O^s
1205 |     - Each NS_j:   independent W_Q^j, W_K^j, W_V^j, W_O^j
1206 |     """
1207 |
1208 |     def __init__(
1209 |         self,
1210 |         d_model: int,
1211 |         num_heads: int,
1212 |         num_ns: int,
1213 |         dropout: float = 0.0,
1214 |     ) -> None:
1215 |         super().__init__()
1216 |         assert d_model % num_heads == 0
1217 |         self.d_model = d_model
```

13. §13 OneTransMixedFFN

行号范围：L1218-L1280（63 行） · **对应论文**：论文 5 OneTrans §Mixed FFN

飞机上读的时候重点关注：

- S 共享 FFN / 每个 NS 独立 FFN
- 参数量: FFN_s (1 份) + FFN_ns[j] (M 份)

```
1218 |         self.num_heads = num_heads
1219 |         self.head_dim = d_model // num_heads
1220 |         self.num_ns = num_ns
1221 |         self.dropout = dropout
1222 |
1223 |         # Shared projections for S-tokens
1224 |         self.W_Q_s = nn.Linear(d_model, d_model, bias=False)
1225 |         self.W_K_s = nn.Linear(d_model, d_model, bias=False)
1226 |         self.W_V_s = nn.Linear(d_model, d_model, bias=False)
1227 |         self.W_O_s = nn.Linear(d_model, d_model)
1228 |
1229 |         # Per-NS-token independent projections
1230 |         self.W_Q_ns = nn.ModuleList(
1231 |             [nn.Linear(d_model, d_model, bias=False) for _ in
range(num_ns)])
1232 |         self.W_K_ns = nn.ModuleList(
1233 |             [nn.Linear(d_model, d_model, bias=False) for _ in
range(num_ns)])
```

```
1234 |         self.W_V_ns = nn.ModuleList(
1235 |             [nn.Linear(d_model, d_model, bias=False) for _ in
range(num_ns)])
1236 |         self.W_O_ns = nn.ModuleList(
1237 |             [nn.Linear(d_model, d_model) for _ in
range(num_ns)])
1238 |
1239 |         # Gating for S output (same pattern as
RoPEMultiheadAttention)
1240 |         self.W_g_s = nn.Linear(d_model, d_model)
1241 |         nn.init.zeros_(self.W_g_s.weight)
1242 |         nn.init.constant_(self.W_g_s.bias, 1.0)
1243 |
1244 |     def _project_ns(
1245 |         self,
1246 |         ns: torch.Tensor,
1247 |         proj_list: nn.ModuleList,
1248 |     ) -> torch.Tensor:
1249 |         """Apply per-token linear to each NS token
independently."""
1250 |         # ns: (B, M, D) → (B, M, D)
1251 |         parts = [proj_list[j](ns[:, j, :]).unsqueeze(1) for j in
range(self.num_ns)]
1252 |         return torch.cat(parts, dim=1)  # (B, M, D)
1253 |
1254 |     @staticmethod
1255 |     def _build_causal_mixed_mask(
1256 |         L: int,
1257 |         M: int,
1258 |         device: torch.device,
1259 |     ) -> torch.Tensor:
1260 |         """Build a bool mask (L+M, L+M) where True = allowed to
attend.
1261 |
1262 |         Pattern:
1263 |           [ones_LxL     |  zeros_LxM  ]
1264 |           [ones_MxL     | causal_MxM  ]
1265 |
1266 |         S tokens use BIDIRECTIONAL attention among themselves
(all S can
1267 |         see all other S). Cross-domain causality would be
arbitrary because
1268 |         S tokens from different domains are concatenated in
alphabet order,
1269 |         not temporal order. NS tokens attend to all S tokens and
causally
1270 |         to previous NS tokens.
1271 |         """
1272 |         total = L + M
1273 |         mask = torch.zeros(total, total, dtype=torch.bool,
device=device)
1274 |         # S-S block: bidirectional (every S sees every S)
1275 |         if L > 0:
1276 |             mask[:L, :L] = True
```

```
1277 |         # NS-S block: every NS sees all S
1278 |         if M > 0 and L > 0:
1279 |             mask[L:, :L] = True
1280 |         # NS-NS block: causal (NS_j sees NS_0 ... NS_j)
```

14. §14 OneTransNSConditioner ★★★ (EXP-013 +0.010 AUC 源头)

行号范围：L1281-L1349（69 行）· **对应论文**：仓库自研扩展（有创新奖潜质）

飞机上读的时候重点关注：

- 在 OneTrans 主 block 前，给 NS token 加序列 mean-pool 信息
- 每个 NS 独立 conditioning FFN（mixed-param 一致）
- v2 无此模块: 0.848712 full train；v3 有: 0.859408 (+0.010)
- 飞机上想清楚：为什么这个能带来 +0.010？类比 HyFormer 的 MultiSeqQueryGenerator（都是给跨特征交互提供 warm-start）

```
1281 |         if M > 0:
1282 |             mask[L:, L:] = torch.tril(
1283 |                 torch.ones(M, M, dtype=torch.bool,
device=device))
1284 |         return mask  # (L+M, L+M)
1285 |
1286 |     def forward(
1287 |         self,
1288 |         s_tokens: torch.Tensor,
1289 |         ns_tokens: torch.Tensor,
1290 |         s_padding_mask: Optional[torch.Tensor] = None,
1291 |     ) -> Tuple[torch.Tensor, torch.Tensor]:
1292 |         """Mixed-param attention over the unified [S | NS]
sequence.
1293 |
1294 |         Args:
1295 |             s_tokens: (B, L, D)  — all domain sequence tokens,
concatenated.
1296 |             ns_tokens: (B, M, D) — NS tokens.
1297 |             s_padding_mask: (B, L) bool, True = padding (masked
out).
1298 |
1299 |         Returns:
1300 |             s_out: (B, L, D)
1301 |             ns_out: (B, M, D)
1302 |         """
1303 |         B, L, D = s_tokens.shape
1304 |         M = self.num_ns
1305 |
1306 |         # ---- QKV projections ----
```

```
1307 |         Q_s = self.W_Q_s(s_tokens)            # (B, L, D)
1308 |         K_s = self.W_K_s(s_tokens)
1309 |         V_s = self.W_V_s(s_tokens)
1310 |
1311 |         Q_ns = self._project_ns(ns_tokens, self.W_Q_ns)   # (B,
M, D)
1312 |         K_ns = self._project_ns(ns_tokens, self.W_K_ns)
1313 |         V_ns = self._project_ns(ns_tokens, self.W_V_ns)
1314 |
1315 |         # Unified sequences
1316 |         Q = torch.cat([Q_s, Q_ns], dim=1)     # (B, L+M, D)
1317 |         K = torch.cat([K_s, K_ns], dim=1)
1318 |         V = torch.cat([V_s, V_ns], dim=1)
1319 |
1320 |         # ---- Reshape for multi-head ----
1321 |         def _to_heads(x: torch.Tensor) -> torch.Tensor:
1322 |             Bx, Lx, _ = x.shape
1323 |             return x.view(Bx, Lx, self.num_heads,
self.head_dim).transpose(1, 2)
1324 |
1325 |         Q = _to_heads(Q)  # (B, H, L+M, head_dim)
1326 |         K = _to_heads(K)
1327 |         V = _to_heads(V)
1328 |
1329 |         # ---- Attention mask ----
1330 |         causal_mixed = self._build_causal_mixed_mask(L, M,
s_tokens.device)
1331 |         # Expand to (B, 1, L+M, L+M)
1332 |         attn_mask = causal_mixed.unsqueeze(0).unsqueeze(0)
1333 |
1334 |         # Incorporate S padding: padded S positions cannot be
attended to.
1335 |         if s_padding_mask is not None:
1336 |             # s_padding_mask: (B, L), True = pad → should NOT be
attended to
1337 |             # Convert to (B, 1, 1, L+M): False where allowed
1338 |             pad_ext = torch.zeros(
1339 |                 B, 1, 1, L + M, dtype=torch.bool,
device=s_tokens.device)
1340 |             pad_ext[:, :, :, :L] =
s_padding_mask.unsqueeze(1).unsqueeze(2)
1341 |             # Combined: allowed = causal_mixed AND NOT padded
1342 |             attn_mask = attn_mask & (~pad_ext)
1343 |
1344 |         dp = self.dropout if self.training else 0.0
1345 |         out = F.scaled_dot_product_attention(
1346 |             Q, K, V, attn_mask=attn_mask, dropout_p=dp)
1347 |         out = torch.nan_to_num(out, nan=0.0)
1348 |         # (B, H, L+M, head_dim) → (B, L+M, D)
1349 |         out = out.transpose(1, 2).contiguous().view(B, L + M, D)
```

15. §15 OneTransBlock

行号范围：L1350-L1484（135 行） · **对应论文**：论文 5 OneTrans (整体装配)

飞机上读的时候重点关注：

- 整体对比 MultiSeqHyFormerBlock 的三步流程
- OneTrans: S+NS 拼成一个 attention 矩阵 + mixed param
- HyFormer: cross-attn 桥接 + RankMixer boost

```
1350 |
1351 |         # ---- Split back + mixed output projection ----
1352 |         out_s_raw = out[:, :L, :]   # (B, L, D)
1353 |         out_ns_raw = out[:, L:, :]  # (B, M, D)
1354 |
1355 |         # S output: shared W_O_s with gating
1356 |         G = torch.sigmoid(self.W_g_s(s_tokens))
1357 |         s_out = self.W_O_s(out_s_raw) * G  # (B, L, D)
1358 |
1359 |         # NS output: per-token W_O_ns
1360 |         ns_parts = [
1361 |             self.W_O_ns[j](out_ns_raw[:, j, :]).unsqueeze(1) for
j in range(M)
1362 |         ]
1363 |         ns_out = torch.cat(ns_parts, dim=1)  # (B, M, D)
1364 |
1365 |         return s_out, ns_out
1366 |
1367 |
1368 | class OneTransMixedFFN(nn.Module):
1369 |     """Mixed-Parameterization Feed-Forward Network.
1370 |
1371 |     S-tokens: shared W1, W2.
1372 |     Each NS_j: independent W1_j, W2_j.
1373 |     """
1374 |
1375 |     def __init__(
1376 |         self,
1377 |         d_model: int,
1378 |         num_ns: int,
1379 |         hidden_mult: int = 4,
1380 |         dropout: float = 0.0,
1381 |     ) -> None:
1382 |         super().__init__()
1383 |         hidden = d_model * hidden_mult
1384 |         self.num_ns = num_ns
1385 |
1386 |         # Shared FFN for S tokens
1387 |         self.ffn_s = nn.Sequential(
1388 |             nn.Linear(d_model, hidden),
1389 |             nn.GELU(),
1390 |             nn.Dropout(dropout),
1391 |             nn.Linear(hidden, d_model),
```

```
1392 |         )
1393 |
1394 |         # Per-NS-token independent FFN
1395 |         self.ffn_ns = nn.ModuleList([
1396 |             nn.Sequential(
1397 |                 nn.Linear(d_model, hidden),
1398 |                 nn.GELU(),
1399 |                 nn.Dropout(dropout),
1400 |                 nn.Linear(hidden, d_model),
1401 |             )
1402 |             for _ in range(num_ns)
1403 |         ])
1404 |
1405 |     def forward(
1406 |         self,
1407 |         s_tokens: torch.Tensor,
1408 |         ns_tokens: torch.Tensor,
1409 |     ) -> Tuple[torch.Tensor, torch.Tensor]:
1410 |         """Apply mixed FFN independently to S and NS tokens.
1411 |
1412 |         Args:
1413 |             s_tokens: (B, L, D)
1414 |             ns_tokens: (B, M, D)
1415 |
1416 |         Returns:
1417 |             s_out: (B, L, D)
1418 |             ns_out: (B, M, D)
1419 |         """
1420 |         s_out = self.ffn_s(s_tokens)
1421 |
1422 |         ns_parts = [
1423 |             self.ffn_ns[j](ns_tokens[:, j, :]).unsqueeze(1)
1424 |             for j in range(self.num_ns)
1425 |         ]
1426 |         ns_out = torch.cat(ns_parts, dim=1)  # (B, M, D)
1427 |
1428 |         return s_out, ns_out
1429 |
1430 |
1431 | class OneTransNSConditioner(nn.Module):
1432 |     """Sequence-conditioned initialization for OneTrans NS
tokens.
1433 |
1434 |     Before the unified attention starts, each NS token receives
an additive
1435 |     update from the mean-pooled domain sequence representations.
This gives
1436 |     NS tokens initial sequence context analogous to HyFormer's
1437 |     MultiSeqQueryGenerator, preventing cold-start of NS → S
attention.
1438 |
1439 |     Each NS token has an independent conditioning FFN (mixed-
parameterization
1440 |     consistent with OneTrans design).
```

```
1441 |     """
1442 |
1443 |     def __init__(
1444 |         self,
1445 |         d_model: int,
1446 |         num_ns: int,
1447 |         num_sequences: int,
1448 |         hidden_mult: int = 2,
1449 |     ) -> None:
1450 |         super().__init__()
1451 |         self.num_ns = num_ns
1452 |         seq_dim = num_sequences * d_model
1453 |         # Pre-LN on the concatenated sequence context
1454 |         self.seq_norm = nn.LayerNorm(seq_dim)
1455 |         # Per-NS-token independent conditioning FFN
1456 |         self.conditioners = nn.ModuleList([
1457 |             nn.Sequential(
1458 |                 nn.Linear(seq_dim, d_model * hidden_mult),
1459 |                 nn.SiLU(),
1460 |                 nn.Linear(d_model * hidden_mult, d_model),
1461 |                 nn.LayerNorm(d_model),
1462 |             )
1463 |             for _ in range(num_ns)
1464 |         ])
1465 |
1466 |     def forward(
1467 |         self,
1468 |         ns_tokens: torch.Tensor,
1469 |         compressed_s_list: list,
1470 |         compressed_mask_list: list,
1471 |     ) -> torch.Tensor:
1472 |         """Add sequence-conditioned delta to each NS token.
1473 |
1474 |         Args:
1475 |             ns_tokens: (B, M, D)
1476 |             compressed_s_list: list of (B, K, D), one per
domain.
1477 |             compressed_mask_list: list of (B, K) bool, True =
padding.
1478 |
1479 |         Returns:
1480 |             ns_tokens updated: (B, M, D)
1481 |         """
1482 |         # Masked mean-pool each domain's compressed S tokens →
(B, D) per domain
1483 |         domain_means = []
1484 |         for s_i, mask_i in zip(compressed_s_list,
compressed_mask_list):
```

16. §16 GroupNSTokenizer (原 baseline 方式)

行号范围：L1485-L1566（82 行） · **对应论文**：—

飞机上读的时候重点关注：

- 怎么把 sparse features 按 group 聚成 NS tokens

```
1485 |             valid = (~mask_i).float().unsqueeze(-1)      # (B,
K, 1)
1486 |             s_mean = (s_i * valid).sum(dim=1) /
valid.sum(dim=1).clamp(min=1)
1487 |             domain_means.append(s_mean)
1488 |
1489 |         seq_ctx = torch.cat(domain_means, dim=-1)        # (B,
num_seq * D)
1490 |         seq_ctx = self.seq_norm(seq_ctx)
1491 |
1492 |         # Per-NS-token update
1493 |         updates = [self.conditioners[j](seq_ctx).unsqueeze(1)
1494 |                    for j in range(self.num_ns)]
1495 |         delta = torch.cat(updates, dim=1)                # (B,
M, D)
1496 |
1497 |         return ns_tokens + delta
1498 |
1499 |
1500 | class OneTransBlock(nn.Module):
1501 |     """Single OneTrans layer (Pre-LN).
1502 |
1503 |     Layout: [compressed_S_tokens | NS_tokens]
1504 |     - Mixed-param attention (bidirectional S, NS sees all S +
causal NS)
1505 |     - Mixed-param FFN (shared for S, independent per NS)
1506 |     - Residual connections + LayerNorm
1507 |
1508 |     The sequence compression (top-K selection per domain) is
handled
1509 |     externally by ``_gather_onetrans_s_tokens`` in
``PCVRHyFormer``.
1510 |     """
1511 |
1512 |     def __init__(
1513 |         self,
1514 |         d_model: int,
1515 |         num_heads: int,
1516 |         num_ns: int,
1517 |         hidden_mult: int = 4,
1518 |         dropout: float = 0.0,
1519 |     ) -> None:
1520 |         super().__init__()
1521 |         self.norm_attn_s = nn.LayerNorm(d_model)
1522 |         self.norm_attn_ns = nn.LayerNorm(d_model)
1523 |         self.attn = OneTransMixedAttn(d_model, num_heads,
num_ns, dropout)
1524 |
1525 |         self.norm_ffn_s = nn.LayerNorm(d_model)
1526 |         self.norm_ffn_ns = nn.LayerNorm(d_model)
```

```
1527 |         self.ffn = OneTransMixedFFN(d_model, num_ns,
hidden_mult, dropout)
1528 |
1529 |     def forward(
1530 |         self,
1531 |         s_tokens: torch.Tensor,
1532 |         ns_tokens: torch.Tensor,
1533 |         s_padding_mask: Optional[torch.Tensor] = None,
1534 |     ) -> Tuple[torch.Tensor, torch.Tensor]:
1535 |         """Apply one OneTrans layer with Pre-LN and residuals.
1536 |
1537 |         Args:
1538 |             s_tokens: (B, L, D) concatenated sequence tokens
from all domains.
1539 |             ns_tokens: (B, M, D) NS tokens (user + item).
1540 |             s_padding_mask: (B, L) bool, True = padding.
1541 |
1542 |         Returns:
1543 |             s_tokens: (B, L, D) updated
1544 |             ns_tokens: (B, M, D) updated
1545 |         """
1546 |         # Pre-LN attention
1547 |         s_normed = self.norm_attn_s(s_tokens)
1548 |         ns_normed = self.norm_attn_ns(ns_tokens)
1549 |         s_attn, ns_attn = self.attn(s_normed, ns_normed,
s_padding_mask)
1550 |
1551 |         s_tokens = s_tokens + s_attn
1552 |         ns_tokens = ns_tokens + ns_attn
1553 |
1554 |         # Pre-LN FFN
1555 |         s_normed = self.norm_ffn_s(s_tokens)
1556 |         ns_normed = self.norm_ffn_ns(ns_tokens)
1557 |         s_ffn, ns_ffn = self.ffn(s_normed, ns_normed)
1558 |
1559 |         s_tokens = s_tokens + s_ffn
1560 |         ns_tokens = ns_tokens + ns_ffn
1561 |
1562 |         return s_tokens, ns_tokens
1563 |
1564 |
1565 |
1566 |     """Single OneTrans layer (Pre-LN).
```

17. §17 RankMixerNSTokenizer (当前默认)

行号范围：L1567-L1688（122 行） · **对应论文**：论文 4 RankMixer NS 部分

飞机上读的时候重点关注：

- 为什么当前默认选它而非 Group
- user_ns_tokens=5, item_ns_tokens=2 的设计动机

```
1567 |
1568 |     Layout: [compressed_S_tokens | NS_tokens]
1569 |     - Mixed-param attention (causal S, NS sees all S + causal
NS)
1570 |     - Mixed-param FFN (shared for S, independent per NS)
1571 |     - Residual connections + LayerNorm
1572 |
1573 |     The sequence compression (top-K selection per domain) is
handled
1574 |     externally by ``_gather_onetrans_s_tokens`` in
``PCVRHyFormer``.
1575 |     """
1576 |
1577 |     def __init__(
1578 |         self,
1579 |         d_model: int,
1580 |         num_heads: int,
1581 |         num_ns: int,
1582 |         hidden_mult: int = 4,
1583 |         dropout: float = 0.0,
1584 |     ) -> None:
1585 |         super().__init__()
1586 |         self.norm_attn_s = nn.LayerNorm(d_model)
1587 |         self.norm_attn_ns = nn.LayerNorm(d_model)
1588 |         self.attn = OneTransMixedAttn(d_model, num_heads,
num_ns, dropout)
1589 |
1590 |         self.norm_ffn_s = nn.LayerNorm(d_model)
1591 |         self.norm_ffn_ns = nn.LayerNorm(d_model)
1592 |         self.ffn = OneTransMixedFFN(d_model, num_ns,
hidden_mult, dropout)
1593 |
1594 |     def forward(
1595 |         self,
1596 |         s_tokens: torch.Tensor,
1597 |         ns_tokens: torch.Tensor,
1598 |         s_padding_mask: Optional[torch.Tensor] = None,
1599 |     ) -> Tuple[torch.Tensor, torch.Tensor]:
1600 |         """Apply one OneTrans layer with Pre-LN and residuals.
1601 |
1602 |         Args:
1603 |             s_tokens: (B, L, D) concatenated sequence tokens
from all domains.
1604 |             ns_tokens: (B, M, D) NS tokens (user + item).
1605 |             s_padding_mask: (B, L) bool, True = padding.
1606 |
1607 |         Returns:
1608 |             s_tokens: (B, L, D) updated
1609 |             ns_tokens: (B, M, D) updated
1610 |         """
1611 |         # Pre-LN attention
1612 |         s_normed = self.norm_attn_s(s_tokens)
1613 |         ns_normed = self.norm_attn_ns(ns_tokens)
1614 |         s_attn, ns_attn = self.attn(s_normed, ns_normed,
```

```
s_padding_mask)
1615 |
1616 |         s_tokens = s_tokens + s_attn
1617 |         ns_tokens = ns_tokens + ns_attn
1618 |
1619 |         # Pre-LN FFN
1620 |         s_normed = self.norm_ffn_s(s_tokens)
1621 |         ns_normed = self.norm_ffn_ns(ns_tokens)
1622 |         s_ffn, ns_ffn = self.ffn(s_normed, ns_normed)
1623 |
1624 |         s_tokens = s_tokens + s_ffn
1625 |         ns_tokens = ns_tokens + ns_ffn
1626 |
1627 |         return s_tokens, ns_tokens
1628 |
1629 |
1630 | #
═══════════════════════════════════════════════════════════════════════
1631 | # PCVRHyFormer Main Model
1632 | #
═══════════════════════════════════════════════════════════════════════
1633 |
1634 |
1635 | class GroupNSTokenizer(nn.Module):
1636 |     """NS tokenizer used by ns_tokenizer_type='group'.
1637 |
1638 |     Groups discrete features by fid, applies shared embedding
with mean
1639 |     pooling per multi-valued feature, then projects each group
to a single
1640 |     NS token (one token per group).
1641 |     """
1642 |
1643 |     def __init__(self, feature_specs: List[Tuple[int, int,
int]],
1644 |                  groups: List[List[int]], emb_dim: int, d_model:
int,
1645 |                  emb_skip_threshold: int = 0) -> None:
1646 |         super().__init__()
1647 |         self.feature_specs = feature_specs
1648 |         self.groups = groups
1649 |         self.emb_dim = emb_dim
1650 |         self.emb_skip_threshold = emb_skip_threshold
1651 |
1652 |         # One embedding table per fid (None if skipped by
emb_skip_threshold
1653 |         # or if vocab_size <= 0 / no vocab info).
1654 |         embs = []
1655 |         for vs, offset, length in feature_specs:
1656 |             skip = int(vs) <= 0 or (emb_skip_threshold > 0 and
int(vs) > emb_skip_threshold)
1657 |             if skip:
1658 |                 embs.append(None)
1659 |             else:
```

```
1660 |                 embs.append(nn.Embedding(int(vs) + 1, emb_dim,
padding_idx=0))
1661 |         self.embs = nn.ModuleList([e for e in embs if e is not
None])
1662 |         # Map from fid index to position in self.embs (or -1 if
filtered)
1663 |         self._emb_index = []
1664 |         real_idx = 0
1665 |         for e in embs:
1666 |             if e is not None:
1667 |                 self._emb_index.append(real_idx)
1668 |                 real_idx += 1
1669 |             else:
1670 |                 self._emb_index.append(-1)
1671 |
1672 |         # Per-group projection: num_fids_in_group * emb_dim ->
d_model (with LayerNorm)
1673 |         self.group_projs = nn.ModuleList([
1674 |             nn.Sequential(
1675 |                 nn.Linear(len(group) * emb_dim, d_model),
1676 |                 nn.LayerNorm(d_model),
1677 |             )
1678 |             for group in groups
1679 |         ])
1680 |
1681 |     def forward(self, int_feats: torch.Tensor) -> torch.Tensor:
1682 |         """Embeds and projects grouped discrete features into NS
tokens.
1683 |
1684 |         Args:
1685 |             int_feats: (B, total_int_dim), concatenated integer
features.
1686 |
1687 |         Returns:
1688 |             Tokens of shape (B, num_groups, D).
```

18. §18 PCVRHyFormer ★ (顶层装配)

行号范围：L1689-L2396 (708 行) · **对应论文**：整体装配 (所有论文的综合体现)

飞机上读的时候重点关注：

- 先过一遍数据流：输入 → NS tokenize → Query gen → blocks → 分类
- 理解 use_onetrans 开关切换 HyFormer vs OneTrans 的位置
- enable_dense_bypass / item_conditioned_query / seq_hash_vocab 扩展参数

```
1689 |         """
1690 |         tokens = []
1691 |         for group, proj in zip(self.groups, self.group_projs):
```

```
1692 |             fid_embs = []
1693 |             for fid_idx in group:
1694 |                 vs, offset, length = self.feature_specs[fid_idx]
1695 |                 emb_real_idx = self._emb_index[fid_idx]
1696 |                 if emb_real_idx == -1:
1697 |                     # Filtered high-cardinality feature: output
zero vector
1698 |                     fid_emb =
int_feats.new_zeros(int_feats.shape[0], self.emb_dim)
1699 |                 else:
1700 |                     emb_layer = self.embs[emb_real_idx]
1701 |                     if length == 1:
1702 |                         # Single-value feature: direct lookup
1703 |                         fid_emb = emb_layer(int_feats[:,
offset].long())  # (B, emb_dim)
1704 |                     else:
1705 |                         # Multi-value feature: lookup then mean
pooling (ignoring padding=0)
1706 |                         vals = int_feats[:, offset:offset +
length].long()  # (B, length)
1707 |                         emb_all = emb_layer(vals)  # (B, length,
emb_dim)
1708 |                         mask = (vals != 0).float().unsqueeze(-1)
# (B, length, 1)
1709 |                         count = mask.sum(dim=1).clamp(min=1)  #
(B, 1)
1710 |                         fid_emb = (emb_all * mask).sum(dim=1) /
count  # (B, emb_dim)
1711 |                 fid_embs.append(fid_emb)
1712 |             cat_emb = torch.cat(fid_embs, dim=-1)  # (B,
num_fids*emb_dim)
1713 |             tokens.append(F.silu(proj(cat_emb)).unsqueeze(1))  #
(B, 1, D)
1714 |         return torch.cat(tokens, dim=1)  # (B, num_groups, D)
1715 |
1716 |
1717 | class RankMixerNSTokenizer(nn.Module):
1718 |     """NS Tokenizer following the RankMixer paper's approach.
1719 |
1720 |     All group embedding vectors are concatenated into a single
long vector,
1721 |     then equally split into num_ns_tokens segments, each
projected to d_model.
1722 |     This allows num_ns_tokens to be chosen freely (independent
of group count).
1723 |     """
1724 |
1725 |     def __init__(
1726 |         self,
1727 |         feature_specs: List[Tuple[int, int, int]],
1728 |         groups: List[List[int]],
1729 |         emb_dim: int,
1730 |         d_model: int,
1731 |         num_ns_tokens: int,
```

```
1732 |         emb_skip_threshold: int = 0,
1733 |     ) -> None:
1734 |         """Initializes RankMixerNSTokenizer.
1735 |
1736 |         Args:
1737 |             feature_specs: [(vocab_size, offset, length), ...]
per feature.
1738 |             groups: List of feature index groups (defines
semantic ordering).
1739 |             emb_dim: Embedding dimension per feature.
1740 |             d_model: Output token dimension.
1741 |             num_ns_tokens: Number of NS tokens to produce (T
segments).
1742 |             emb_skip_threshold: Skip embedding for features with
vocab > threshold.
1743 |         """
1744 |         super().__init__()
1745 |         self.feature_specs = feature_specs
1746 |         self.groups = groups
1747 |         self.emb_dim = emb_dim
1748 |         self.num_ns_tokens = num_ns_tokens
1749 |         self.emb_skip_threshold = emb_skip_threshold
1750 |
1751 |         # One embedding table per fid (None if skipped by
emb_skip_threshold
1752 |         # or if vocab_size <= 0 / no vocab info).
1753 |         embs = []
1754 |         for vs, offset, length in feature_specs:
1755 |             skip = int(vs) <= 0 or (emb_skip_threshold > 0 and
int(vs) > emb_skip_threshold)
1756 |             if skip:
1757 |                 embs.append(None)
1758 |             else:
1759 |                 embs.append(nn.Embedding(int(vs) + 1, emb_dim,
padding_idx=0))
1760 |         self.embs = nn.ModuleList([e for e in embs if e is not
None])
1761 |         # Map from fid index to position in self.embs (or -1 if
filtered)
1762 |         self._emb_index = []
1763 |         real_idx = 0
1764 |         for e in embs:
1765 |             if e is not None:
1766 |                 self._emb_index.append(real_idx)
1767 |                 real_idx += 1
1768 |             else:
1769 |                 self._emb_index.append(-1)
1770 |
1771 |         # Compute total embedding dim: sum of all fids across
all groups
1772 |         total_num_fids = sum(len(g) for g in groups)
1773 |         total_emb_dim = total_num_fids * emb_dim
1774 |
1775 |         # Pad total_emb_dim to be divisible by num_ns_tokens
```

```
1776 |         self.chunk_dim = math.ceil(total_emb_dim /
num_ns_tokens)
1777 |         self.padded_total_dim = self.chunk_dim * num_ns_tokens
1778 |         self._pad_size = self.padded_total_dim - total_emb_dim
1779 |
1780 |         # Per-chunk projection: chunk_dim -> d_model with
LayerNorm
1781 |         self.token_projs = nn.ModuleList([
1782 |             nn.Sequential(
1783 |                 nn.Linear(self.chunk_dim, d_model),
1784 |                 nn.LayerNorm(d_model),
1785 |             )
1786 |             for _ in range(num_ns_tokens)
1787 |         ])
1788 |
1789 |         logging.info(
1790 |             f"RankMixerNSTokenizer: {total_num_fids} fids, "
1791 |             f"total_emb_dim={total_emb_dim},
chunk_dim={self.chunk_dim}, "
1792 |             f"num_ns_tokens={num_ns_tokens},
pad={self._pad_size}"
1793 |         )
1794 |
1795 |     def forward(self, int_feats: torch.Tensor) -> torch.Tensor:
1796 |         """Embeds all features, concatenates, splits, and
projects.
1797 |
1798 |         Args:
1799 |             int_feats: (B, total_int_dim) concatenated integer
features.
1800 |
1801 |         Returns:
1802 |             (B, num_ns_tokens, d_model) tensor.
1803 |         """
1804 |         # 1. Embed all fids in group order → flat cat
1805 |         all_embs = []
1806 |         for group in self.groups:
1807 |             for fid_idx in group:
1808 |                 vs, offset, length = self.feature_specs[fid_idx]
1809 |                 emb_real_idx = self._emb_index[fid_idx]
1810 |                 if emb_real_idx == -1:
1811 |                     fid_emb =
int_feats.new_zeros(int_feats.shape[0], self.emb_dim)
1812 |                 else:
1813 |                     emb_layer = self.embs[emb_real_idx]
1814 |                     if length == 1:
1815 |                         fid_emb = emb_layer(int_feats[:,
offset].long())
1816 |                     else:
1817 |                         vals = int_feats[:, offset:offset +
length].long()
1818 |                         emb_all = emb_layer(vals)
1819 |                         mask = (vals != 0).float().unsqueeze(-1)
1820 |                         count = mask.sum(dim=1).clamp(min=1)
```

```
1821 |                         fid_emb = (emb_all * mask).sum(dim=1) /
count
1822 |                 all_embs.append(fid_emb)
1823 |
1824 |         cat_emb = torch.cat(all_embs, dim=-1)  # (B,
total_emb_dim)
1825 |
1826 |         # 2. Pad if needed
1827 |         if self._pad_size > 0:
1828 |             cat_emb = F.pad(cat_emb, (0, self._pad_size))  # (B,
padded_total_dim)
1829 |
1830 |         # 3. Split into num_ns_tokens chunks and project each
1831 |         chunks = cat_emb.split(self.chunk_dim, dim=-1)  # list
of (B, chunk_dim)
1832 |         tokens = []
1833 |         for chunk, proj in zip(chunks, self.token_projs):
1834 |             tokens.append(F.silu(proj(chunk)).unsqueeze(1))  #
(B, 1, d_model)
1835 |
1836 |         return torch.cat(tokens, dim=1)  # (B, num_ns_tokens,
d_model)
1837 |
1838 |
1839 | class PCVRHyFormer(nn.Module):
1840 |     """PCVRHyFormer model for post-click conversion rate
prediction.
1841 |
1842 |     Combines MultiSeqHyFormerBlock and MultiSeqQueryGenerator to
process
1843 |     multiple input sequences with non-sequence features.
1844 |     """
1845 |
1846 |     def __init__(
1847 |         self,
1848 |         # Data schema
1849 |         user_int_feature_specs: List[Tuple[int, int, int]],
1850 |         item_int_feature_specs: List[Tuple[int, int, int]],
1851 |         user_dense_dim: int,
1852 |         item_dense_dim: int,
1853 |         seq_vocab_sizes: "dict[str, List[int]]",  # {domain:
[vocab_size_per_fid, ...]}
1854 |         # NS grouping config (grouped by fid index)
1855 |         user_ns_groups: List[List[int]],
1856 |         item_ns_groups: List[List[int]],
1857 |         # Model hyperparameters
1858 |         d_model: int = 64,
1859 |         emb_dim: int = 64,
1860 |         num_queries: int = 1,
1861 |         num_hyformer_blocks: int = 2,
1862 |         num_heads: int = 4,
1863 |         seq_encoder_type: str = 'transformer',
1864 |         hidden_mult: int = 4,
1865 |         dropout_rate: float = 0.01,
```

```
1866 |         seq_top_k: int = 50,
1867 |         seq_causal: bool = False,
1868 |         action_num: int = 1,
1869 |         num_time_buckets: int = 65,
1870 |         rank_mixer_mode: str = 'full',
1871 |         # T19 / ADR-007: FFN parameterization inside
RankMixerBlock.
1872 |         #   'shared'    = one FFN shared across T tokens
(legacy, default).
1873 |         #   'per_token' = independent FFN weights per token
(RankMixer paper
1874 |         #                 original definition). ~T× dense params
per block.
1875 |         rank_mixer_ffn_mode: str = 'shared',
1876 |         use_rope: bool = False,
1877 |         rope_base: float = 10000.0,
1878 |         emb_skip_threshold: int = 0,
1879 |         seq_id_threshold: int = 10000,
1880 |         # NS tokenizer variant
1881 |         ns_tokenizer_type: str = 'rankmixer',
1882 |         user_ns_tokens: int = 0,
1883 |         item_ns_tokens: int = 0,
1884 |         # OneTrans unified attention (replaces HyFormer query-
decoding blocks)
1885 |         use_onetrans: bool = False,
1886 |         onetrans_top_k: int = 50,
1887 |         # T13: hash embedding for sequence features with vocab >
emb_skip_threshold
1888 |         # (0 = disabled, use zero vector for skipped features as
before)
1889 |         seq_hash_vocab: int = 0,
1890 |         # Item-conditioned query generation (DIN-style target-
item awareness)
1891 |         item_conditioned_query: bool = False,
1892 |         # ADR-004 · Target-item hard interaction injected into
seq tokens
1893 |         # (seq-token level target-item conditioning, orthogonal
to ICQ which
1894 |         # is query-side). Default 'off' keeps behaviour
identical to baseline.
1895 |         target_item_seq_injection: str = 'off',
1896 |         target_inject_alpha_init: float = 0.0,
1897 |         # Dense bypass: project dense features directly to the
classifier input,
1898 |         # bypassing NS-token dilution in HyFormer blocks. When
enabled the dense
1899 |         # token is removed from ns_tokens and the classifier
input dimension is
1900 |         # extended by d_model (d_model + d_model = 2*d_model).
1901 |         enable_dense_bypass: bool = False,
1902 |     ) -> None:
1903 |         super().__init__()
1904 |
1905 |         self.d_model = d_model
```

```
1906 |         self.emb_dim = emb_dim
1907 |         self.action_num = action_num
1908 |         self.num_queries = num_queries
1909 |         self.seq_domains = sorted(seq_vocab_sizes.keys())  #
deterministic order
1910 |         self.enable_dense_bypass = enable_dense_bypass
1911 |         self.num_sequences = len(self.seq_domains)
1912 |         self.num_time_buckets = num_time_buckets
1913 |         self.rank_mixer_mode = rank_mixer_mode
1914 |         self.rank_mixer_ffn_mode = rank_mixer_ffn_mode
1915 |         self.use_rope = use_rope
1916 |         self.emb_skip_threshold = emb_skip_threshold
1917 |         self.seq_id_threshold = seq_id_threshold
1918 |         self.ns_tokenizer_type = ns_tokenizer_type
1919 |         self.use_onetrans = use_onetrans
1920 |         self.onetrans_top_k = onetrans_top_k
1921 |         self.seq_hash_vocab = seq_hash_vocab
1922 |         self.target_item_seq_injection =
target_item_seq_injection
1923 |         self.target_inject_alpha_init = target_inject_alpha_init
1924 |
1925 |         # ================== NS Tokens Construction
==================
1926 |
1927 |         if ns_tokenizer_type == 'group':
1928 |             # Original: one NS token per group
1929 |             self.user_ns_tokenizer = GroupNSTokenizer(
1930 |                 feature_specs=user_int_feature_specs,
1931 |                 groups=user_ns_groups,
1932 |                 emb_dim=emb_dim,
1933 |                 d_model=d_model,
1934 |                 emb_skip_threshold=emb_skip_threshold,
1935 |             )
1936 |             num_user_ns = len(user_ns_groups)
1937 |
1938 |             self.item_ns_tokenizer = GroupNSTokenizer(
1939 |                 feature_specs=item_int_feature_specs,
1940 |                 groups=item_ns_groups,
1941 |                 emb_dim=emb_dim,
1942 |                 d_model=d_model,
1943 |                 emb_skip_threshold=emb_skip_threshold,
1944 |             )
1945 |             num_item_ns = len(item_ns_groups)
1946 |         elif ns_tokenizer_type == 'rankmixer':
1947 |             # RankMixer paper style: all embeddings cat → split
→ project
1948 |             # 0 means auto: fall back to group count
1949 |             if user_ns_tokens <= 0:
1950 |                 user_ns_tokens = len(user_ns_groups)
1951 |             if item_ns_tokens <= 0:
1952 |                 item_ns_tokens = len(item_ns_groups)
1953 |             self.user_ns_tokenizer = RankMixerNSTokenizer(
1954 |                 feature_specs=user_int_feature_specs,
1955 |                 groups=user_ns_groups,
```

```
1956 |                 emb_dim=emb_dim,
1957 |                 d_model=d_model,
1958 |                 num_ns_tokens=user_ns_tokens,
1959 |                 emb_skip_threshold=emb_skip_threshold,
1960 |             )
1961 |             num_user_ns = user_ns_tokens
1962 |
1963 |             self.item_ns_tokenizer = RankMixerNSTokenizer(
1964 |                 feature_specs=item_int_feature_specs,
1965 |                 groups=item_ns_groups,
1966 |                 emb_dim=emb_dim,
1967 |                 d_model=d_model,
1968 |                 num_ns_tokens=item_ns_tokens,
1969 |                 emb_skip_threshold=emb_skip_threshold,
1970 |             )
1971 |             num_item_ns = item_ns_tokens
1972 |         else:
1973 |             raise ValueError(f"Unknown ns_tokenizer_type:
{ns_tokenizer_type}")
1974 |
1975 |         # Save for downstream modules that need to slice item NS
tokens out
1976 |         # of the concatenated ns_tokens tensor (e.g. ICQ, target
injector).
1977 |         self.num_item_ns = num_item_ns
1978 |         self.num_user_ns = num_user_ns
1979 |
1980 |         # User dense feature projection (if available)
1981 |         self.has_user_dense = user_dense_dim > 0
1982 |         if self.has_user_dense:
1983 |             self.user_dense_proj = nn.Sequential(
1984 |                 nn.Linear(user_dense_dim, d_model),
1985 |                 nn.LayerNorm(d_model),
1986 |             )
1987 |
1988 |         # Item dense feature projection (if available)
1989 |         self.has_item_dense = item_dense_dim > 0
1990 |         if self.has_item_dense:
1991 |             self.item_dense_proj = nn.Sequential(
1992 |                 nn.Linear(item_dense_dim, d_model),
1993 |                 nn.LayerNorm(d_model),
1994 |             )
1995 |
1996 |         # Total NS token count: bypass mode keeps the dense NS
token so that
1997 |         # the rank_mixer T-divisibility constraint (d_model % T
== 0) is
1998 |         # preserved. The bypass projection adds a SECOND,
undiluted path for
1999 |         # the dense features directly into the classifier — they
flow through
2000 |         # both the NS-token path and the bypass path
simultaneously.
2001 |         self.num_ns = (num_user_ns + (1 if self.has_user_dense
```

```
else 0)
2002 |                         + num_item_ns + (1 if self.has_item_dense
else 0))
2003 |
2004 |         # Dense bypass projections: an extra projection added
ALONGSIDE the
2005 |         # NS-token path when enable_dense_bypass=True. The
projected vector is
2006 |         # concatenated directly to the output of output_proj
before the
2007 |         # classifier, giving the dense features a full-strength
signal path
2008 |         # independent of the HyFormer block dilution.
2009 |         self.has_dense_bypass = enable_dense_bypass and
self.has_user_dense
2010 |         if self.has_dense_bypass:
2011 |             self.user_dense_bypass_proj = nn.Sequential(
2012 |                 nn.Linear(user_dense_dim, d_model),
2013 |                 nn.LayerNorm(d_model),
2014 |                 nn.SiLU(),
2015 |             )
2016 |
2017 |         # ================== Check d_model % T == 0 constraint
(full mode only) ==================
2018 |         T = num_queries * self.num_sequences + self.num_ns
2019 |         if rank_mixer_mode == 'full' and d_model % T != 0 and
not use_onetrans:
2020 |             valid_T_values = [t for t in range(1, d_model + 1)
if d_model % t == 0]
2021 |             raise ValueError(
2022 |                 f"d_model={d_model} must be divisible by
T=num_queries*num_sequences+num_ns="
2023 |                 f"{num_queries}
*{self.num_sequences}+{self.num_ns}={T}. "
2024 |                 f"Valid T values for d_model={d_model}:
{valid_T_values}"
2025 |             )
2026 |
2027 |         # ================== Seq Tokens Embedding
==================
2028 |         # seq_id_threshold decides which features inside the seq
tokenizer are
2029 |         # treated as id features (they receive extra dropout).
It is fully
2030 |         # independent of emb_skip_threshold (which skips
Embedding creation).
2031 |         self.seq_id_emb_dropout = nn.Dropout(dropout_rate * 2)
2032 |
2033 |         def _make_seq_embs(vocab_sizes):
2034 |             """Create embedding list for sequence domain
features.
2035 |
2036 |             Features with vocab_size <= 0 → None (zero vector).
2037 |             Features with vocab_size > emb_skip_threshold → None
```

```
unless
2038 |             seq_hash_vocab > 0, in which case a hash embedding
of size
2039 |             seq_hash_vocab is created and values are hashed
modulo
2040 |             seq_hash_vocab at forward time (T13 fix for
domain_c_seq_47).
2041 |             """
2042 |             embs_raw = []
2043 |             hash_flags = []
2044 |             for vs in vocab_sizes:
2045 |                 if int(vs) <= 0:
2046 |                     embs_raw.append(None)
2047 |                     hash_flags.append(False)
2048 |                 elif emb_skip_threshold > 0 and int(vs) >
emb_skip_threshold:
2049 |                     if seq_hash_vocab > 0:
2050 |                         # Hash embedding: recover high-vocab
item IDs via modulo
2051 |                         embs_raw.append(nn.Embedding(
2052 |                             int(seq_hash_vocab) + 1, emb_dim,
padding_idx=0))
2053 |                         hash_flags.append(True)
2054 |                     else:
2055 |                         embs_raw.append(None)
2056 |                         hash_flags.append(False)
2057 |                 else:
2058 |                     embs_raw.append(nn.Embedding(int(vs) + 1,
emb_dim, padding_idx=0))
2059 |                     hash_flags.append(False)
2060 |             module_list = nn.ModuleList([e for e in embs_raw if
e is not None])
2061 |             index_map = []
2062 |             real_idx = 0
2063 |             for e in embs_raw:
2064 |                 if e is not None:
2065 |                     index_map.append(real_idx)
2066 |                     real_idx += 1
2067 |                 else:
2068 |                     index_map.append(-1)
2069 |             is_id = [int(vs) > seq_id_threshold for vs in
vocab_sizes]
2070 |             return module_list, index_map, is_id, hash_flags
2071 |
2072 |         # ================== Dynamic Sequence Embeddings
==================
2073 |         self._seq_embs = nn.ModuleDict()
2074 |         self._seq_emb_index = {}    # domain -> index_map
2075 |         self._seq_is_id = {}        # domain -> is_id list
2076 |         self._seq_hash_flags = {}   # domain -> hash_flags (True
= modulo hash)
2077 |         self._seq_vocab_sizes = {}  # domain -> vocab_sizes list
2078 |         self._seq_proj = nn.ModuleDict()
2079 |
```

```
2080 |         for domain in self.seq_domains:
2081 |             vs = seq_vocab_sizes[domain]
2082 |             embs, idx_map, is_id, hash_flags =
_make_seq_embs(vs)
2083 |             self._seq_embs[domain] = embs
2084 |             self._seq_emb_index[domain] = idx_map
2085 |             self._seq_is_id[domain] = is_id
2086 |             self._seq_hash_flags[domain] = hash_flags
2087 |             self._seq_vocab_sizes[domain] = vs
2088 |             self._seq_proj[domain] = nn.Sequential(
2089 |                 nn.Linear(len(vs) * emb_dim, d_model),
2090 |                 nn.LayerNorm(d_model),
2091 |             )
2092 |
2093 |         # ================== Time Interval Bucket Embedding
(optional) ==================
2094 |         if num_time_buckets > 0:
2095 |             self.time_embedding = nn.Embedding(num_time_buckets,
d_model, padding_idx=0)
2096 |
2097 |         # ================== HyFormer OR OneTrans Components
==================
2098 |         # Target-item seq injector (ADR-004) — applied to
seq_tokens_list
2099 |         # BEFORE the HyFormer / OneTrans path, so it benefits
both. When
2100 |         # target_item_seq_injection='off' this is a zero-param
identity.
2101 |         self.target_item_seq_injector = TargetItemSeqInjector(
2102 |             d_model=d_model,
2103 |             num_sequences=self.num_sequences,
2104 |             mode=target_item_seq_injection,
2105 |             alpha_init=target_inject_alpha_init,
2106 |         )
2107 |
2108 |         if not use_onetrans:
2109 |             # ---- Classic HyFormer path ----
2110 |             self.query_generator = MultiSeqQueryGenerator(
2111 |                 d_model=d_model,
2112 |                 num_ns=self.num_ns,
2113 |                 num_queries=num_queries,
2114 |                 num_sequences=self.num_sequences,
2115 |                 hidden_mult=hidden_mult,
2116 |                 item_conditioned=item_conditioned_query,
2117 |                 num_item_ns=num_item_ns,
2118 |             )
2119 |             self.blocks = nn.ModuleList([
2120 |                 MultiSeqHyFormerBlock(
2121 |                     d_model=d_model,
2122 |                     num_heads=num_heads,
2123 |                     num_queries=num_queries,
2124 |                     num_ns=self.num_ns,
2125 |                     num_sequences=self.num_sequences,
2126 |                     seq_encoder_type=seq_encoder_type,
```

```
2127 |                     hidden_mult=hidden_mult,
2128 |                     dropout=dropout_rate,
2129 |                     top_k=seq_top_k,
2130 |                     causal=seq_causal,
2131 |                     rank_mixer_mode=rank_mixer_mode,
2132 |                     rank_mixer_ffn_mode=rank_mixer_ffn_mode,
2133 |                 )
2134 |                 for _ in range(num_hyformer_blocks)
2135 |             ])
2136 |             # Output: concat Q tokens from all domains → project
to d_model
2137 |             self.output_proj = nn.Sequential(
2138 |                 nn.Linear(num_queries * self.num_sequences *
d_model, d_model),
2139 |                 nn.LayerNorm(d_model),
2140 |             )
2141 |         else:
2142 |             # ---- OneTrans path ----
2143 |             # Per-domain lightweight SwiGLU encoder (processes
sequences before
2144 |             # top-K compression; shared across OneTrans blocks
for efficiency)
2145 |             self.ot_seq_encoders = nn.ModuleDict({
2146 |                 domain: SwiGLUEncoder(d_model, hidden_mult,
dropout_rate)
2147 |                 for domain in self.seq_domains
2148 |             })
2149 |             # Stack of OneTrans blocks
2150 |             self.onetrans_blocks = nn.ModuleList([
2151 |                 OneTransBlock(
2152 |                     d_model=d_model,
2153 |                     num_heads=num_heads,
2154 |                     num_ns=self.num_ns,
2155 |                     hidden_mult=hidden_mult,
2156 |                     dropout=dropout_rate,
2157 |                 )
2158 |                 for _ in range(num_hyformer_blocks)
2159 |             ])
2160 |             # NS-token sequence conditioner: gives NS tokens
initial sequence
2161 |             # context before the first unified attention
(prevents cold-start)
2162 |             self.ot_ns_conditioner = OneTransNSConditioner(
2163 |                 d_model=d_model,
2164 |                 num_ns=self.num_ns,
2165 |                 num_sequences=self.num_sequences,
2166 |                 hidden_mult=2,
2167 |             )
2168 |             # Output: NS tokens flattened → project to d_model
2169 |             self.output_proj = nn.Sequential(
2170 |                 nn.Linear(self.num_ns * d_model, d_model),
2171 |                 nn.LayerNorm(d_model),
2172 |             )
2173 |             # No query_generator or HyFormer blocks needed
```

```
2174 |             self.query_generator = None
2175 |             self.blocks = nn.ModuleList()
2176 |
2177 |         # ================== RoPE (HyFormer path only)
==================
2178 |         if use_rope and not use_onetrans:
2179 |             head_dim = d_model // num_heads
2180 |             self.rotary_emb = RotaryEmbedding(dim=head_dim,
base=rope_base)
2181 |         else:
2182 |             self.rotary_emb = None
2183 |
2184 |         # Dropout
2185 |         self.emb_dropout = nn.Dropout(dropout_rate)
2186 |
2187 |         # Classifier input dimension: d_model normally;
d_model*2 when dense
2188 |         # bypass is active because the bypass vector is
concatenated to output.
2189 |         clsfier_in_dim = d_model * 2 if self.has_dense_bypass
else d_model
2190 |
2191 |         # Classifier
2192 |         self.clsfier = nn.Sequential(
2193 |             nn.Linear(clsfier_in_dim, d_model),
2194 |             nn.LayerNorm(d_model),
2195 |             nn.SiLU(),
2196 |             nn.Dropout(dropout_rate),
2197 |             nn.Linear(d_model, action_num)
2198 |         )
2199 |
2200 |         # Initialize parameters
2201 |         self._init_params()
2202 |
2203 |         # Log emb_skip_threshold filtering stats
2204 |         if emb_skip_threshold > 0:
2205 |             def _count_filtered(vocab_sizes, emb_index):
2206 |                 filtered = sum(1 for idx in emb_index if idx ==
-1)
2207 |                 return filtered, len(vocab_sizes)
2208 |             for domain in self.seq_domains:
2209 |                 f, t =
_count_filtered(self._seq_vocab_sizes[domain],
self._seq_emb_index[domain])
2210 |                 if f > 0:
2211 |
logging.info(f"emb_skip_threshold={emb_skip_threshold}: {domain}
skipped {f}/{t} features")
2212 |             for name, tokenizer in [
2213 |                 ("user_ns", self.user_ns_tokenizer),
2214 |                 ("item_ns", self.item_ns_tokenizer),
2215 |             ]:
2216 |                 f = sum(1 for idx in tokenizer._emb_index if idx
== -1)
```

```
2217 |                 t = len(tokenizer._emb_index)
2218 |                 if f > 0:
2219 |
logging.info(f"emb_skip_threshold={emb_skip_threshold}: {name} skipped
{f}/{t} features")
2220 |
2221 |     def _init_params(self) -> None:
2222 |         """Applies Xavier initialization to all embedding
weights."""
2223 |         for domain in self.seq_domains:
2224 |             for emb in self._seq_embs[domain]:
2225 |                 nn.init.xavier_normal_(emb.weight.data)
2226 |                 emb.weight.data[0, :] = 0
2227 |
2228 |         for tokenizer in [self.user_ns_tokenizer,
self.item_ns_tokenizer]:
2229 |             for emb in tokenizer.embs:
2230 |                 nn.init.xavier_normal_(emb.weight.data)
2231 |                 emb.weight.data[0, :] = 0
2232 |
2233 |         if self.num_time_buckets > 0:
2234 |
nn.init.xavier_normal_(self.time_embedding.weight.data)
2235 |             self.time_embedding.weight.data[0, :] = 0
2236 |
2237 |     def reinit_high_cardinality_params(
2238 |         self, cardinality_threshold: int = 10000
2239 |     ) -> "set[int]":
2240 |         """Reinitializes only high-cardinality embeddings.
2241 |
2242 |         Preserves low-cardinality and time feature embeddings.
2243 |
2244 |         Args:
2245 |             cardinality_threshold: Only embeddings with
vocab_size exceeding
2246 |                 this value are reinitialized.
2247 |
2248 |         Returns:
2249 |             A set of data_ptr() values for reinitialized
parameters.
2250 |         """
2251 |         reinit_count = 0
2252 |         skip_count = 0
2253 |         reinit_ptrs = set()
2254 |
2255 |         for emb_list, vocab_sizes, emb_index in [
2256 |             (self._seq_embs[d], self._seq_vocab_sizes[d],
self._seq_emb_index[d])
2257 |             for d in self.seq_domains
2258 |         ]:
2259 |             for i, vs in enumerate(vocab_sizes):
2260 |                 real_idx = emb_index[i]
2261 |                 if real_idx == -1:
2262 |                     # Skipped by emb_skip_threshold, no
```

```
embedding to reinit
2263 |                     continue
2264 |                 emb = emb_list[real_idx]
2265 |                 if int(vs) > cardinality_threshold:
2266 |                     nn.init.xavier_normal_(emb.weight.data)
2267 |                     emb.weight.data[0, :] = 0
2268 |                     reinit_ptrs.add(emb.weight.data_ptr())
2269 |                     reinit_count += 1
2270 |                 else:
2271 |                     skip_count += 1
2272 |
2273 |         for tokenizer, specs in [
2274 |             (self.user_ns_tokenizer,
self.user_ns_tokenizer.feature_specs),
2275 |             (self.item_ns_tokenizer,
self.item_ns_tokenizer.feature_specs),
2276 |         ]:
2277 |             for i, (vs, offset, length) in enumerate(specs):
2278 |                 real_idx = tokenizer._emb_index[i]
2279 |                 if real_idx == -1:
2280 |                     continue
2281 |                 emb = tokenizer.embs[real_idx]
2282 |                 if int(vs) > cardinality_threshold:
2283 |                     nn.init.xavier_normal_(emb.weight.data)
2284 |                     emb.weight.data[0, :] = 0
2285 |                     reinit_ptrs.add(emb.weight.data_ptr())
2286 |                     reinit_count += 1
2287 |                 else:
2288 |                     skip_count += 1
2289 |
2290 |         # time_embedding is always preserved
2291 |         if self.num_time_buckets > 0:
2292 |             skip_count += 1
2293 |
2294 |         logging.info(f"Re-initialized {reinit_count} high-
cardinality Embeddings "
2295 |                      f"(vocab>{cardinality_threshold}), kept
{skip_count}")
2296 |         return reinit_ptrs
2297 |
2298 |     def get_sparse_params(self) -> List[nn.Parameter]:
2299 |         """Returns all embedding table parameters (optimized
with Adagrad)."""
2300 |         sparse_params = set()
2301 |         for module in self.modules():
2302 |             if isinstance(module, nn.Embedding):
2303 |                 sparse_params.add(module.weight.data_ptr())
2304 |         return [p for p in self.parameters() if p.data_ptr() in
sparse_params]
2305 |
2306 |     def get_dense_params(self) -> List[nn.Parameter]:
2307 |         """Returns all non-embedding parameters (optimized with
AdamW)."""
2308 |         sparse_ptrs = {p.data_ptr() for p in
```

```
self.get_sparse_params()}
2309 |         return [p for p in self.parameters() if p.data_ptr() not
in sparse_ptrs]
2310 |
2311 |     def _embed_seq_domain(
2312 |         self,
2313 |         seq: torch.Tensor,
2314 |         sideinfo_embs: nn.ModuleList,
2315 |         proj: nn.Module,
2316 |         is_id: List[bool],
2317 |         emb_index: List[int],
2318 |         time_bucket_ids: torch.Tensor,
2319 |         hash_flags: Optional[List[bool]] = None,
2320 |     ) -> torch.Tensor:
2321 |         """Embeds a sequence domain by concatenating sideinfo
embeddings and projecting to d_model."""
2322 |         B, S, L = seq.shape
2323 |         emb_list = []
2324 |         for i in range(S):
2325 |             real_idx = emb_index[i] if i < len(emb_index) else
-1
2326 |             if real_idx == -1:
2327 |                 emb_list.append(seq.new_zeros(B, L,
self.emb_dim, dtype=torch.float))
2328 |             else:
2329 |                 emb = sideinfo_embs[real_idx]
2330 |                 vals = seq[:, i, :]  # (B, L)
2331 |                 # Hash embedding: map high-vocab item IDs via
modulo
2332 |                 if hash_flags is not None and i <
len(hash_flags) and hash_flags[i]:
2333 |                     # Non-zero values get hashed; zero (padding)
stays 0
2334 |                     nonzero = vals != 0
2335 |                     vals = vals.clone()
2336 |                     vals[nonzero] = (vals[nonzero] %
self.seq_hash_vocab).clamp(min=1)
2337 |                 e = emb(vals)  # (B, L, emb_dim)
2338 |                 if is_id[i] and self.training:
2339 |                     e = self.seq_id_emb_dropout(e)
2340 |                 emb_list.append(e)
2341 |         cat_emb = torch.cat(emb_list, dim=-1)  # (B, L,
S*emb_dim)
2342 |         token_emb = F.gelu(proj(cat_emb))  # (B, L, D)
2343 |
2344 |         if self.num_time_buckets > 0:
2345 |             token_emb = token_emb +
self.time_embedding(time_bucket_ids)
2346 |
2347 |         return token_emb
2348 |
2349 |     def _make_padding_mask(
2350 |         self, seq_len: torch.Tensor, max_len: int
2351 |     ) -> torch.Tensor:
```

```
2352 |         """Generates a padding mask from sequence lengths."""
2353 |         device = seq_len.device
2354 |         idx = torch.arange(max_len, device=device).unsqueeze(0)
# (1, max_len)
2355 |         return idx >= seq_len.unsqueeze(1)  # (B, max_len)
2356 |
2357 |     def _gather_onetrans_topk(
2358 |         self,
2359 |         seq_tokens: torch.Tensor,
2360 |         padding_mask: torch.Tensor,
2361 |     ) -> Tuple[torch.Tensor, torch.Tensor]:
2362 |         """Take the last top_k valid tokens per sample (like
LongerEncoder).
2363 |
2364 |         Args:
2365 |             seq_tokens: (B, L, D)
2366 |             padding_mask: (B, L) True = padding
2367 |
2368 |         Returns:
2369 |             topk_tokens: (B, K, D)
2370 |             topk_mask: (B, K) True = padding
2371 |         """
2372 |         B, L, D = seq_tokens.shape
2373 |         K = min(self.onetrans_top_k, L)
2374 |         valid_len = (~padding_mask).sum(dim=1)  # (B,)
2375 |         actual_k = valid_len.clamp(max=K)
2376 |         start_pos = (valid_len - actual_k).clamp(min=0)
2377 |
2378 |         device = seq_tokens.device
2379 |         offsets = torch.arange(K,
device=device).unsqueeze(0).expand(B, -1)
2380 |         indices = (start_pos.unsqueeze(1) + offsets).clamp(0, L
- 1)
2381 |         indices_exp = indices.unsqueeze(-1).expand(-1, -1, D)
2382 |         topk_tokens = torch.gather(seq_tokens, 1, indices_exp)
# (B, K, D)
2383 |
2384 |         pos_idx = torch.arange(K, device=device).unsqueeze(0)
2385 |         topk_mask = pos_idx >= actual_k.unsqueeze(1)  # (B, K)
True = pad
2386 |         topk_tokens = topk_tokens *
(~topk_mask).unsqueeze(-1).float()
2387 |         return topk_tokens, topk_mask
2388 |
2389 |     def _run_onetrans_blocks(
2390 |         self,
2391 |         seq_tokens_list: list,
2392 |         seq_masks_list: list,
2393 |         ns_tokens: torch.Tensor,
2394 |         apply_dropout: bool = True,
2395 |     ) -> torch.Tensor:
2396 |         """OneTrans forward path.
```